Banco de Dados Relacional

Linguagem de Banco de Dados

Prof. Me. Fred Lucena

Plano de Aula



Objetivo:
Levar ao aluno os conhecimentos para o uso de linguagem para manipulação de banco de dados.

Metodologia:
Apresentação teórica sobre o assunto.

Recursos Didáticos:
Slides eletrônicos e atividades em sala.

Formas de Avaliação:
Questionário on-line no Kahoot

Bibliografia Básica

COUGO, Paulo. **Modelagem conceitual e projeto de banco de dados.** 3.ed. Rio de Janeiro: F & W Publications, 1997. 284p.

HEUSER, Carlos Alberto. **Projeto de Banco de Dados.** 4.ed. Rio Grande do Sul: Editora Sagra, 1997. 205p.

MACHADO, Felipe Nery Rodrigues; ABREU, Maurício Pereira de. **Projeto de banco de dados: uma visão prática.** 16.ed. São Paulo: Érica, 2002. 320p.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistemas de Gerenciamento de Banco de Dados.** 3ed. São Paulo: Mc Graw Hill, 2008. 884p.

SETZER, Valdemar W; SILVA, Flávio Soares Corrêa da. **Bancos de dados: Aprenda o que São, Melhore seu Conhecimento, Construa os seus.** 1.ed. São Paulo: Edgard Blucher, 2005. 390p.

SILBERSCHATZ, Abraham; KORTH, Henry F; SUDARSHAN, S. **Sistema de banco de dados.** 3.ed. São Paulo: Campus, 2006. 808p.

Projeto de Banco de Dados

Conceitos Básicos

Prof. Me. Fred Lucena

O que é um projeto de banco de dados?

É a descrição detalhada de um plano para a criação de um banco de dados. Seu desenvolvimento é dividido em quatro etapas:

- Levantamento e Análise de Requisitos;
- Modelagem Conceitual;
- Modelagem Lógica;
- Modelagem Física.



Projeto de Banco de Dados

Levantamento e Análise de Requisitos

Prof. Me. Fred Lucena

O que é Levantamento e Análise de Requisitos?

É a atividade através da qual se procura entender as regras de negócio do universo a ser modelado e por meio da qual será desenhado o que se costuma chamar de minimundo ou universo de discurso.

Os requisitos são classificados em:

- Requisitos Funcionais;
- Requisitos Não-Funcionais.



Obs. Engenharia de Requisitos é o conjunto formado pelo “Levantamento”, “Análise” e “Documentação” de requisitos.

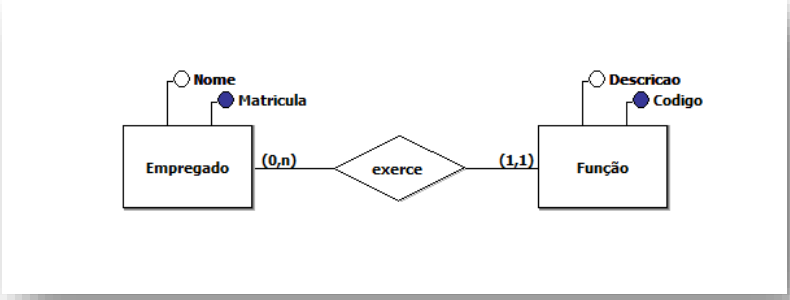
Projeto de Banco de Dados

Modelagem Conceitual

Prof. Me. Fred Lucena

O que é Modelagem Conceitual?

É a atividade através da qual é criado um modelo abstrato que descreve como os dados deverão ser agrupados e relacionados em um banco. É gerado a partir do levantamento e análise dos requisitos e representado, graficamente, pelo DER (Diagrama de Entidades e Relacionamentos).



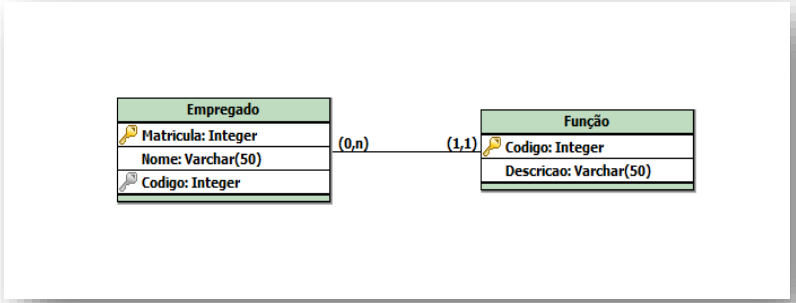
Projeto de Banco de Dados

Modelagem Lógica

Prof. Me. Fred Lucena

O que é Modelagem Lógica?

É a atividade através da qual é criado um modelo detalhado que define as nomenclaturas, atributos e chaves das tabelas em um banco de dados. É gerado a partir do modelo conceitual e representado, graficamente, pelo Modelo Lógico.



Projeto de Banco de Dados

Modelagem Física

Prof. Me. Fred Lucena

O que é Modelagem Física?

É a atividade através da qual é definida toda a estrutura física do banco de dados. É gerado a partir do modelo lógico e representado através de códigos SQL.

```
CREATE TABLE Empregado ( Matricula Integer PRIMARY KEY,
                         Nome Varchar(50),
                         Codigo Integer );

CREATE TABLE Função (Codigo Integer PRIMARY KEY,
                     Descricao Varchar(50) );

ALTER TABLE Empregado ADD FOREIGN KEY(Codigo) REFERENCES Função (Codigo)
```

Projeto de Banco de Dados

Elementos de Banco de Dados

Prof. Me. Fred Lucena

Elementos de Banco de Dados

Um Banco de Dados Relacional é basicamente composto por dois elementos:

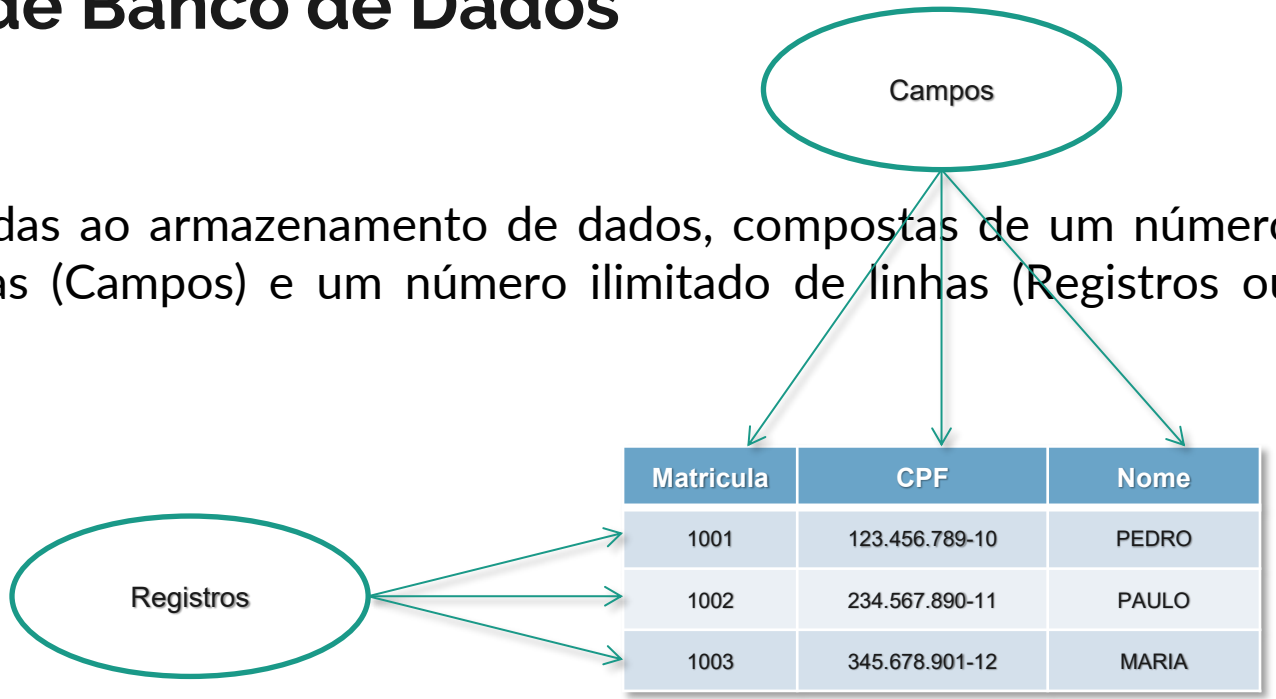
- Tabelas;
- Chaves.



Elementos de Banco de Dados

Tabelas:

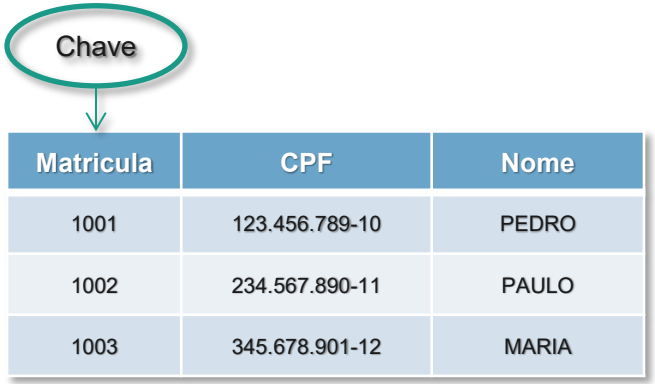
Estruturas destinadas ao armazenamento de dados, compostas de um número limitado de colunas (Campos) e um número ilimitado de linhas (Registros ou Tuplas).



Elementos de Banco de Dados

Chaves:

Campo, ou conjunto de campos, com o objetivo de identificar unicamente um registro e ou estabelecer relações entre registros.



Elementos de Banco de Dados

Chaves Candidatas ou Alternativas:

Campo, ou conjunto de campos, que podem identificar unicamente um registro.

Matricula	CPF	RG	OrgExp	Nome	CodDpt
1001	123.456.789-10	1.234.567	SDS-PE	PEDRO	101
1002	234.567.890-11	2.345.678	SDS-PE	PAULO	102
1003	345.678.901-12	1.234.567	SDS-SP	MARIA	103
1004	456.789.012-13	3.456.789	SDS-PE	CARLOS	102
1005	567.890.123-14	1.234.567	SDS-RJ	MARIA	103

Elementos de Banco de Dados

Chaves Candidatas ou Alternativas:

Campos, ou conjunto de campos, que podem identificar unicamente um registro.

Matricula	CPF	RG	OrgExp	Nome	CodDpt
1001	123.456.789-10	1.234.567	SDS-PE	PEDRO	101
1002	234.567.890-11	2.345.678	SDS-PE	PAULO	102
1003	345.678.901-12	1.234.567	SDS-SP	MARIA	103
1004	456.789.012-13	3.456.789	SDS-PE	CARLOS	102
1005	567.890.123-14	1.234.567	SDS-RJ	MARIA	103

Elementos de Banco de Dados

Chaves Candidatas ou Alternativas:

Campos, ou conjunto de campos, que podem identificar unicamente um registro.

Matricula	CPF	RG	OrgExp	Nome	CodDpt
1001	123.456.789-10	1.234.567	SDS-PE	PEDRO	101
1002	234.567.890-11	2.345.678	SDS-PE	PAULO	102
1003	345.678.901-12	1.234.567	SDS-SP	MARIA	103
1004	456.789.012-13	3.456.789	SDS-PE	CARLOS	102
1005	567.890.123-14	1.234.567	SDS-RJ	MARIA	103

Elementos de Banco de Dados

Chaves Primárias:

Chaves candidatas escolhidas como identificadoras de registros em suas tabelas.

Matricula	CPF	RG	OrgExp	Nome	CodDpt
1001	123.456.789-10	1.234.567	SDS-PE	PEDRO	101
1002	234.567.890-11	2.345.678	SDS-PE	PAULO	102
1003	345.678.901-12	1.234.567	SDS-SP	MARIA	103
1004	456.789.012-13	3.456.789	SDS-PE	CARLOS	102
1005	567.890.123-14	1.234.567	SDS-RJ	MARIA	103

Elementos de Banco de Dados

Chaves Estrangeiras:

Campo, ou conjunto de campos, de uma tabela que se liga a chave primaria de uma tabela relacionada.

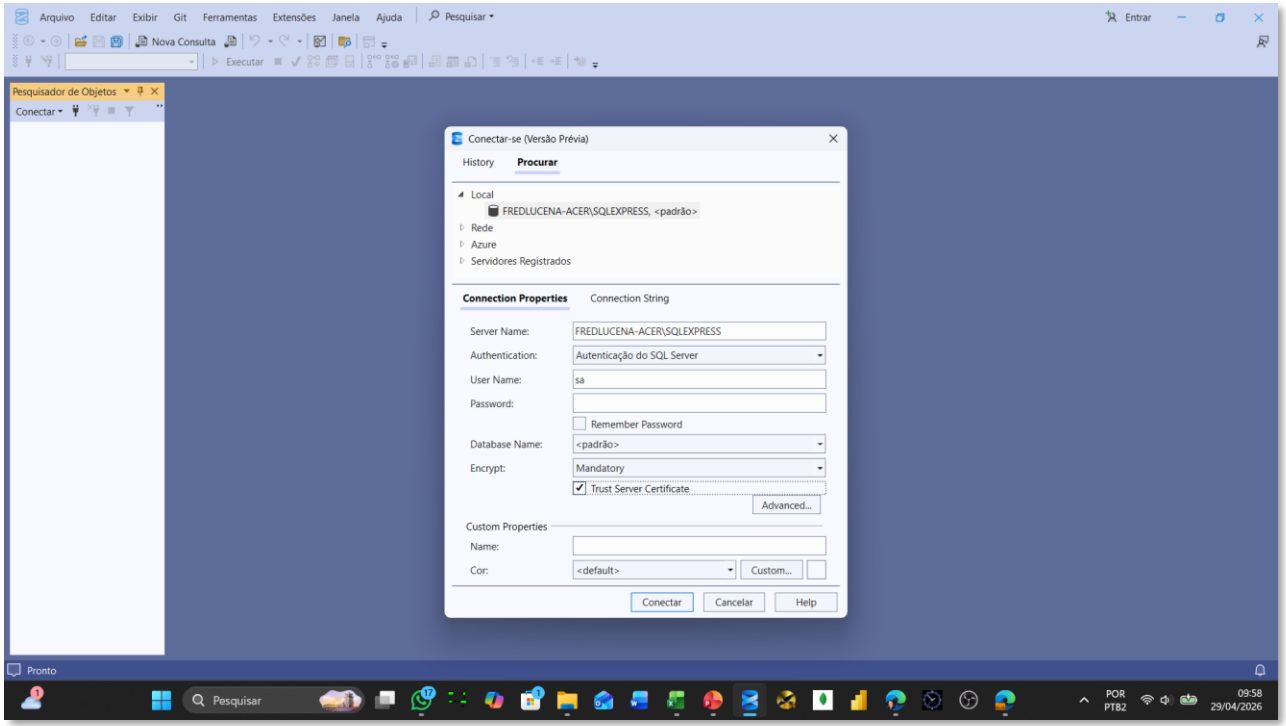
Matricula	CPF	RG	OrgExp	Nome	CodDpto
1001	123.456.789-10	1.234.567	SDS-PE	PEDRO	101
1002	234.567.890-11	2.345.678	SDS-PE	PAULO	102

CodDpto	Departamento
101	Pessoal
102	Financeiro

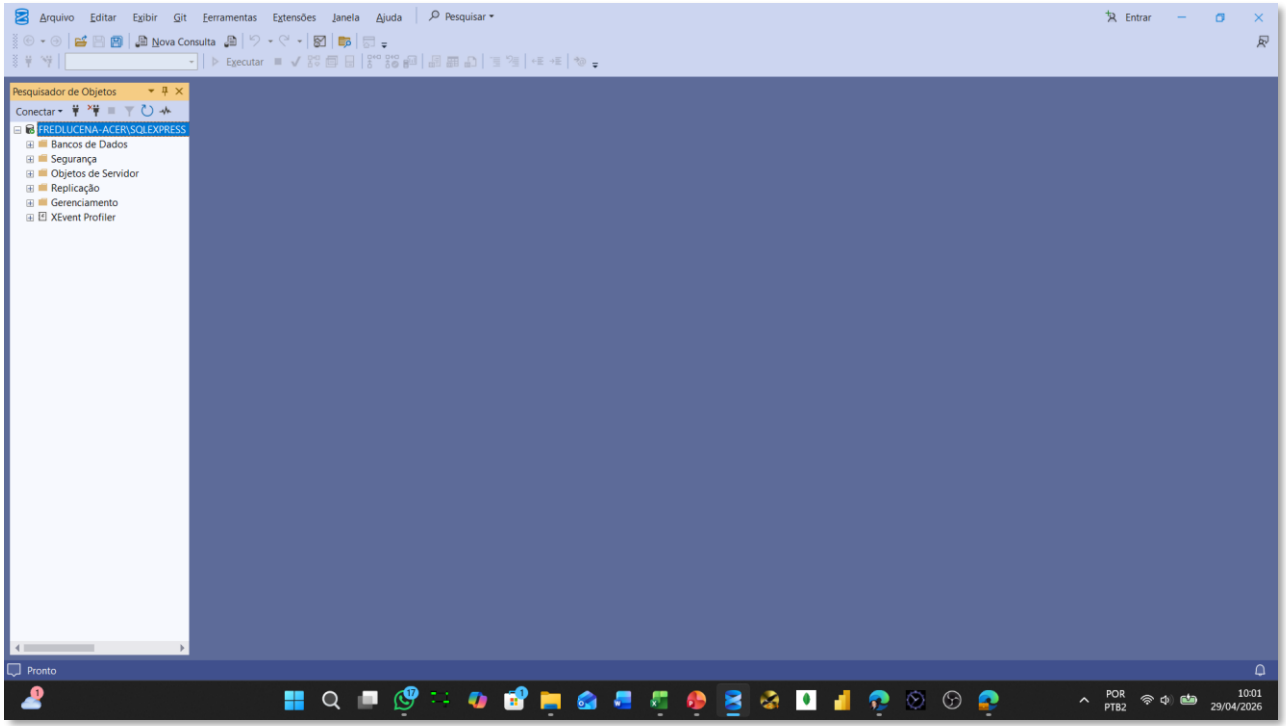
O SQL Server Management Studio

Prof. Me. Fred Lucena

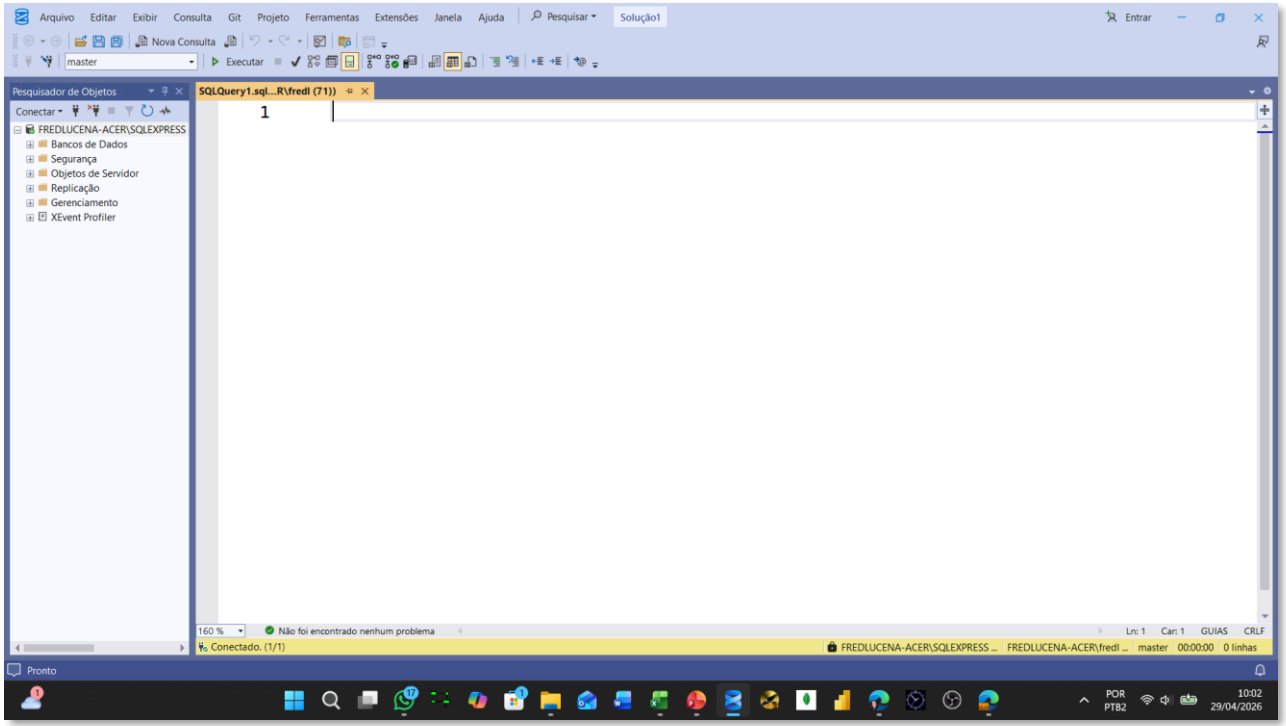
O SQL SERVER MANAGEMENT STUDIO



O SQL SERVER MANAGEMENT STUDIO



O SQL SERVER MANAGEMENT STUDIO

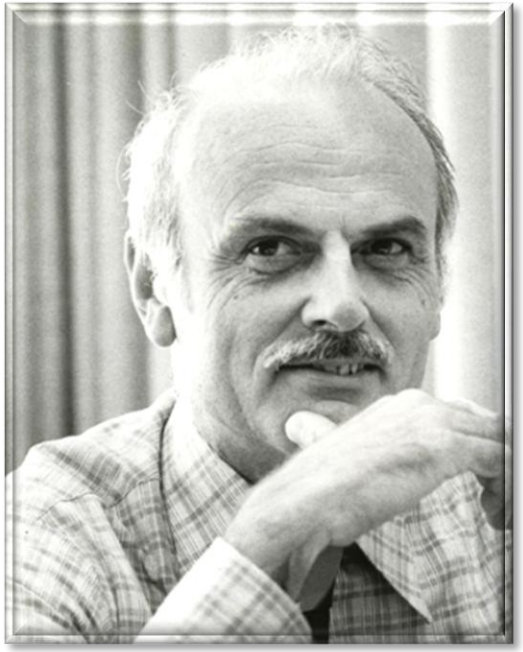


CIÊNCIA DA COMPUTAÇÃO
Linguagem SQL

Prof. Me. Fred Lucena

O Modelo Relacional

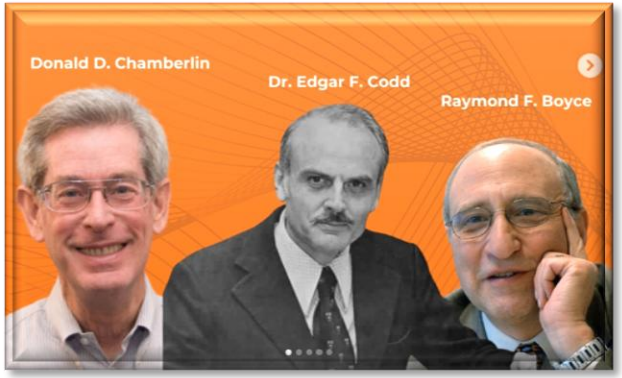
O modelo relacional foi proposto em 1970 pelo matemático, cientista da computação e pesquisador britânico Edgar Frank Codd no artigo “A Relational Model of Data for Large Shared Data Banks”.



Edgar Frank Codd

Linguagem SQL

A linguagem SQL foi desenvolvida em 1974 por Donald D. Chamberlin e Raymond F. Boyce, pesquisadores da IBM.



Linguagem SQL

O primeiro nome da linguagem era SEQUEL (Structured English Query Language). Ela foi desenvolvida como parte do projeto System R da IBM, que buscava implementar os conceitos do modelo relacional. Posteriormente, o nome passou a ser apenas SQL (Structured English Query Language) por questões de marca registrada.

Linguagem SQL

O SQL, embora padronizado pela ANSI e ISO, possui muitas variações e extensões produzidos pelos diferentes fabricantes de sistemas gerenciadores de bases de dados. Tipicamente a linguagem pode ser migrada de plataforma para plataforma sem mudanças estruturais principais.

(http://pt.wikipedia.org/wiki/SQL).

Linha do tempo resumida:

- 1970 → Edgar Frank Codd propõe o modelo relacional.
- 1974 → Chamberlin e Boyce desenvolvem o SEQUEL/SQL.
- 1979 → Surge o primeiro banco comercial com SQL, o Oracle Database.
- 1986 → SQL torna-se padrão ANSI.
- 1987 → SQL torna-se padrão ISO.

Linguagem SQL – Comandos

Prof. Me. Fred Lucena

Comandos DDL – Data Definition Language

Refere-se ao conjunto de comandos utilizados para incluir, alterar e excluir objetos (bancos de dados, tabelas, visões, etc.) no SGBD.

Comandos DML – Data Manipulation Language

Refere-se ao conjunto de comandos utilizados para incluir, alterar e excluir dados.

Comandos DCL – Data Control Language

Refere-se ao conjunto de comandos utilizados para controlar o nível de acesso de usuários.

Comandos DQL – Data Query Language

Refere-se ao comando SELECT utilizado basicamente na execução de consultas.

Comandos DTL – Data Transaction Language

Refere-se ao conjunto de comandos utilizados para iniciar, confirmar ou desfazer uma transação.

Comandos DDL

Prof. Me. Fred Lucena

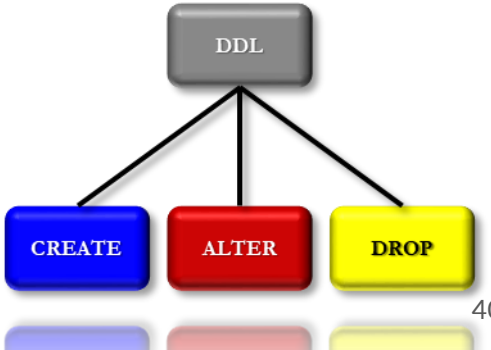
O que são os comandos DDL?

São os comandos destinados a criar, alterar e excluir objetos no SGBD.

Os objetos são (bancos de dados, tabelas, visões, índices, etc.)

Os principais comandos desta categoria são:

CREATE, ALTER e DROP

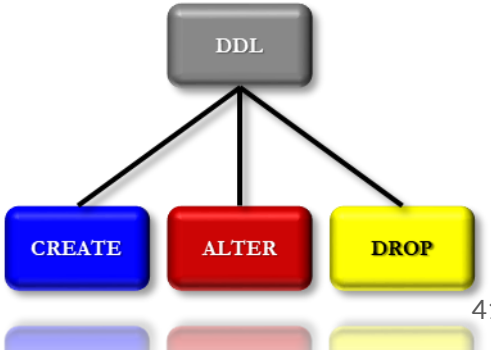


Sintaxe dos comandos DDL em banco de dados:

CREATE DATABASE <nome_banco>;

ALTER DATABASE <nome_banco> **MODIFY NAME** = <novo_nome>;

DROP DATABASE <nome_banco>;



Comandos DDL em banco de dados:

CREATE DATABASE meubanco;

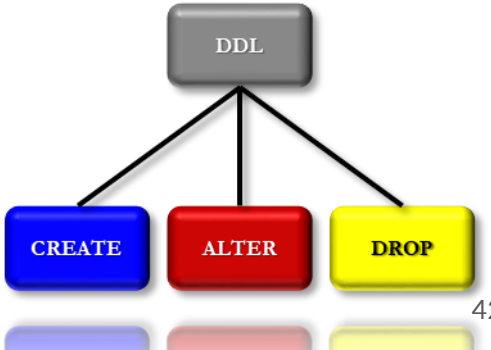
USE meubanco;

ALTER DATABASE meubanco **MODIFY NAME** = meu_banco;

USE master;

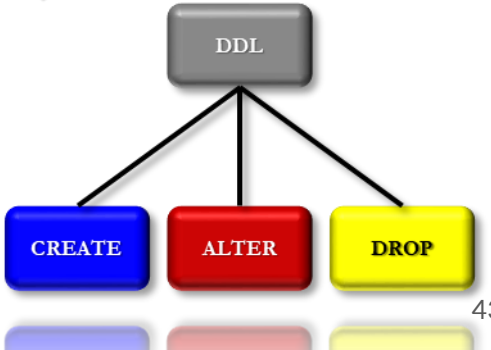
DROP DATABASE meu_banco;

- PostgreSQL: **RENAME TO**
- MySQL : **NÃO PERMITE**



Exercícios:

1) Criar um banco com seu nome;
2) Entrar no banco criado;
3) Renomear o banco criado para teste;
4) Excluir o banco teste.

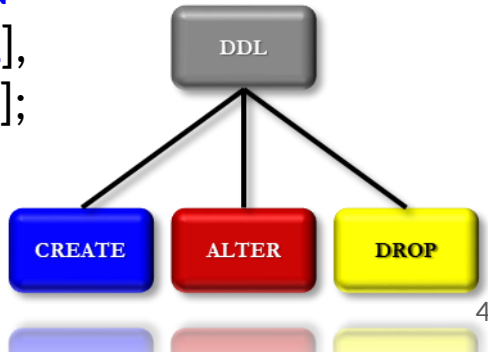


Sintaxe dos comandos DDL em tabelas:

CREATE TABLE <tabela> (<campo1> <tipo> [NOT NULL] ,
<campo2> <tipo> [NOT NULL] ,
[PRIMARY KEY (campo1,campo2,...),
[FOREIGN KEY (campo) REFERENCES tabela2);

ALTER TABLE <tabela> ADD/DROP/ALTER COLUMN
<campo1> <tipo> [NOT NULL],
<campon> <tipo> [NOT NULL];

DROP TABLE <tabela>;



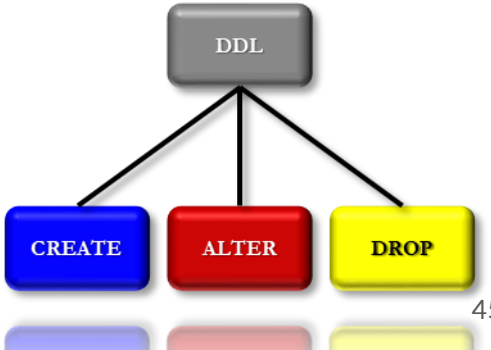
Comandos DDL em tabelas:

```
CREATE DATABASE BancoDados;
GO
USE BancoDados;
CREATE TABLE Departamento (
                    DeptNume    INTEGER,
                    DeptNome    VARCHAR(20),
                    DeptLocal   VARCHAR(20),
                    DeptOrcam   DECIMAL(12,2)
);

SELECT * FROM Departamento;
```

```
SQL SEVER - ORACLE

  INTEGER     => NUMBER(n)
VARCHAR(n)    => VARCHAR2(n)
DECIMAL(n,d)=> NUMBER(n,d)
```



Comandos DDL em tabelas:

No Oracle

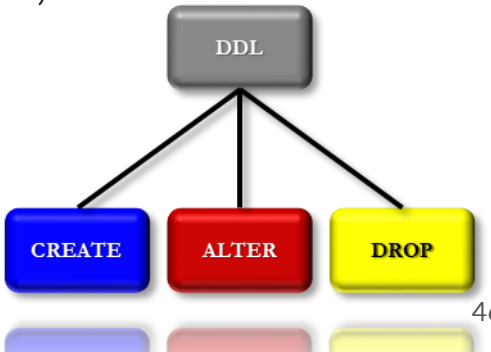
MODIFY

```
ALTER TABLE Departamento ADD DeptSala CHAR(2);

ALTER TABLE Departamento ALTER COLUMN DeptNome VARCHAR (50);

ALTER TABLE Departamento DROP COLUMN DeptSala;

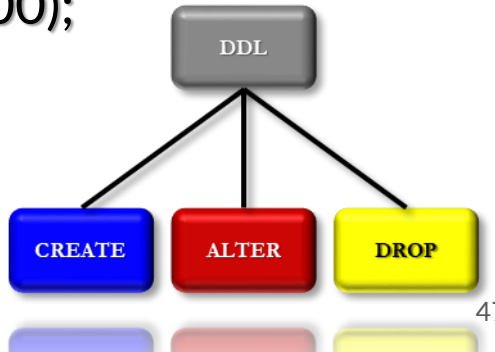
DROP TABLE Departamento;
```



Exercícios:

1) Criar banco de dados de nome “**Banco**”;
2) Posicionar no banco criado;
3) Renomear o banco para “**Agenda**”;
4) Criar a tabela ao lado com o nome **PESSOAL**;
5) Adicionar o campo **OBSERVACAO VARCHAR**(200);
6) Alterar o tamanho de **OBSERVACAO** para 150;
7) Excluir o campo **OBSERVACAO;**
8) Excluir a tabela Pessoal.

CAMPO	DOMÍNIO
MATRICULA	INTEGER
NOME	VARCHAR(50)
NASCIMENTO	DATE
SEXO	CHAR(1)
SALARIO	DECIMAL(10,2)



Exercícios:

1) Criar banco de dados de nome “**Banco**”;

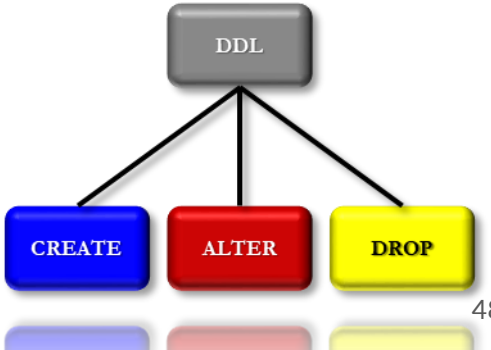
```
CREATE DATABASE Banco;
```

2) Posicionar no banco criado;

```
USE Banco;
```

3) Renomear o banco para “Agenda”;

```
ALTER DATABASE Banco
MODIFY NAME = Agenda;
```

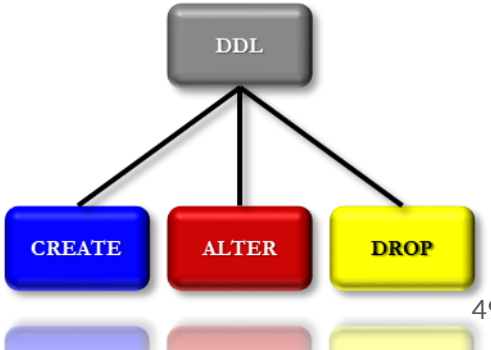


Exercícios:

CAMPO	DOMÍNIO
MATRICULA	INTEGER
NOME	VARCHAR(50)
NASCIMENTO	DATE
SEXO	CHAR(1)
SALARIO	DECIMAL(10,2)

4) Criar a tabela ao lado com o nome **PESSOAL**;

```
CREATE TABLE Pessoal
            ( Matricula   INTEGER,
              Nome        VARCHAR(50),
              Nascimento  DATE,
              Sexo        CHAR(1),
              Salario     DECIMAL(10,2) );
```



Exercícios:

5) Adicionar o campo Observacao VARCHAR(200);

```
ALTER TABLE Pessoal ADD Observacao VARCHAR(200);
```

6) Alterar o tamanho do campo Observacao para 150;

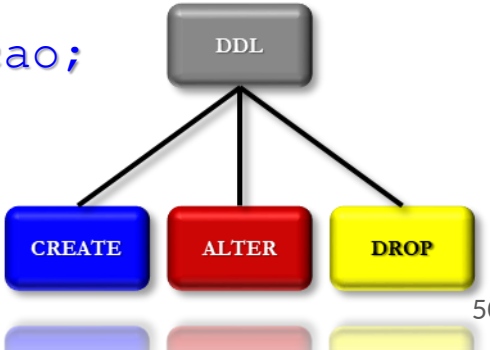
```
ALTER TABLE Pessoal ALTER COLUMN Observacao VARCHAR(150);
```

7) Excluir o campo **Observacao**.

```
ALTER TABLE Pessoal DROP COLUMN Observacao;
```

8) Excluir a Tabela **Pessoal**.

```
DROP TABLE Pessoal;
```



Pesquisa:

Pesquise quais os tipos de dados são mais indicados para os atributos abaixo e justifique:

CAMPO	DOMÍNIO	JUSTIFICATIVA
Codigo	INTEGER	Normalmente os códigos são números inteiros.
Nome	VARCHAR(50)	Nomes não têm tamanho definido.
Nascimento	DATE	Nascimento se refere a uma data.
Hora	TIME	Hora se refere a um horário.
Sexo	CHAR(1)	Sexo têm tamanho definido.
Salario	DECIMAL(10,2)	Salario é um valor monetário.

O que é um índice?

Um índice em um banco de dados é uma estrutura de dados que melhora a velocidade das operações de consulta, como busca, filtragem, ordenação e junção. Ele funciona de maneira semelhante ao índice de um livro, onde você pode rapidamente localizar informações específicas sem precisar percorrer todo o conteúdo.

Como o índice funciona?

Exemplo básico

Imagine uma tabela `cliente` com os seguintes dados:

id_cliente	nome	idade
1	João	25
2	Maria	30
3	Carlos	22
4	Ana	28

Se você criar um índice na coluna `nome`, o índice pode armazenar algo assim:

nome	id_cliente
Ana	4
Carlos	3
João	1
Maria	2

Criando um Índice

```
CREATE INDEX in_clienteAlfa ON cliente(nome)
```

Comandos DML

Prof. Me. Fred Lucena

O que são os comandos DML?

São os comandos destinados a inserir, alterar e excluir dados em tabelas.

Os principais comandos desta categoria são:

INSERT, UPDATE e DELETE

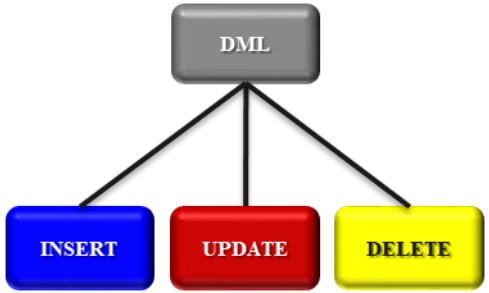


Sintaxe dos comandos DML:

```
INSERT INTO <tabela> (<campos>) VALUES (<valores>);

UPDATE <tabela> SET <campo> = <expressão> [WHERE <condição>];

DELETE FROM <tabela> [WHERE <condição>];
```



Inserindo dados:

```
INSERT INTO <Tabela>
        (Campo_1, Campo_2, Campo_n)
VALUES (Valor_1, Valor_2, Valor_n);
------------------------ OU ------------------------
INSERT INTO <Tabela>

VALUES (Valor_1, Valor_2, Valor_n);
```

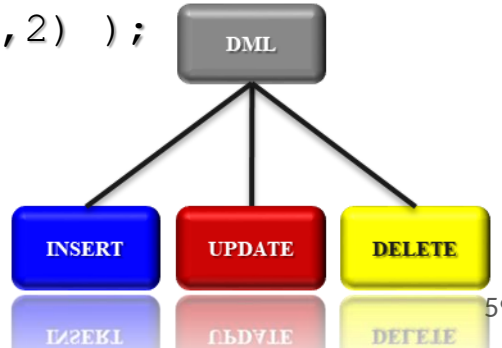


Exercícios:

1) Criar banco de dados de nome “Banco”;
2) Posicionar no banco criado;
3) Criar a tabela abaixo;

```
CREATE TABLE Departamento( Numero     INTEGER,
                           Nome       VARCHAR(50),
                           Cidade     VARCHAR(20),
                           Orcamento  DECIMAL(12,2) );
```

```
SELECT * FROM Departamento;
```



Exercícios:

4) Inserir os dados abaixo;

```
INSERT INTO Departamento
            ( Numero, Nome, Cidade, Orcamento )
VALUES
            ( 70, 'Producao', 'Recife', 10000 );

SELECT * FROM Departamento;
```



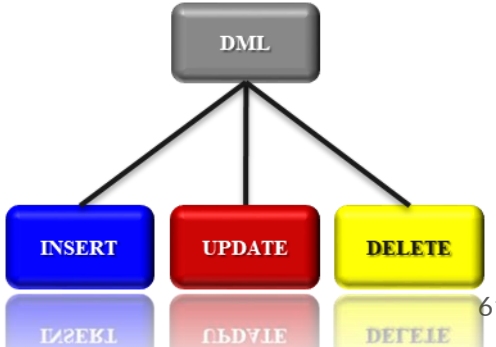
Exercícios:

4) Alterar o departamento de Producao para Pessoal;

```
UPDATE Departamento
SET    Nome = 'Pessoal'
WHERE  Nome = 'Producao';
```

Sem a cláusula **WHERE**, a alteração afetará todos os registros.

```
SELECT * FROM Departamento;
```



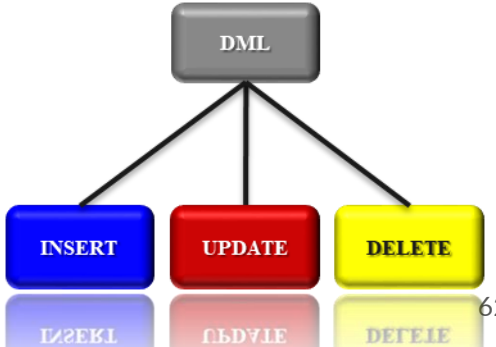
Exercícios:

5) Exclua os dados da tabela Departamento onde o departamento for igual a pessoal;

```
DELETE FROM Departamento
WHERE  Nome = 'Pessoal';
```

Sem a cláusula **WHERE**, todos os registros serão excluídos.

```
SELECT * FROM Departamento;
```



Exercícios:

6) Inclua os dados abaixo na tabela Departamento;

```
INSERT INTO departamento
            ( Numero, Nome, Cidade, Orcamento )
VALUES
            ( 10, 'Produção', 'Recife', 15000 ),
            ( 20, 'Contábil', 'Olinda', 30000 ),
            ( 30, 'Contábil', 'Olinda', 40000 ),
            ( 40, 'Contábil', 'Recife', 50000 );

SELECT * FROM Departamento;
```

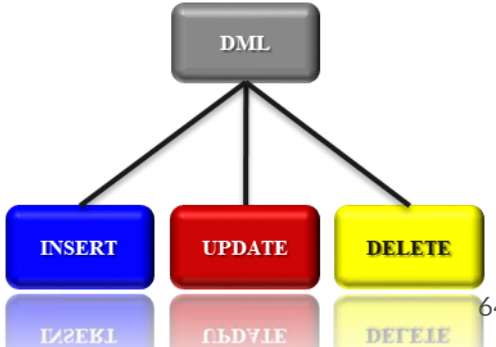


Exercícios:

7) Altere o nome dos departamentos para Estoque onde o nome for igual a Contábil e tiver Orcamento >= 35000;

```
UPDATE Departamento
SET    Nome = 'Estoque'
WHERE  Nome = 'Contábil' AND Orcamento >= 35000;

SELECT * FROM Departamento;
```

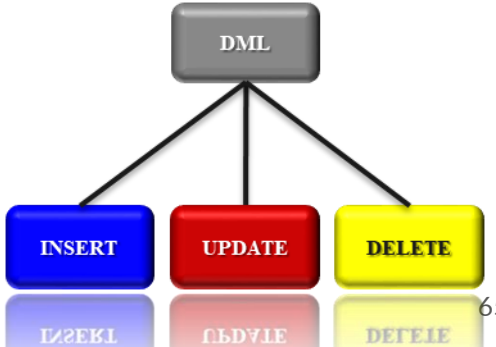


Exercícios:

8) Exclua os dados onde o Local = 'Recife' ou o Orcamento >= 40000;

```
DELETE FROM departamento
WHERE  Cidade = 'Recife' OR Orcamento >= 40000;
```

```
SELECT * FROM Departamento;
```



Restrição de Domínio

Prof. Me. Fred Lucena

O que é domínio?

É o conjunto de todos os valores possíveis para um determinado atributo.

Ex:

INTEGER, CHAR(), VARCHAR(), DATE(), TIME(), etc.

O que é restrição de domínio?

É uma limitação aplicada sobre o domínio do atributo.

Ex:

NOT NULL, **PRIMARY KEY**, **FOREIGN KEY**, **CHECK**, **etc.**

Restrição NOT NULL

Prof. Me. Fred Lucena

NOT NULL

Torna o preenchimento do atributo obrigatório.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Sx_Pessoal CHAR(1) );
-------------------------------------------------------------
INSERT INTO Pessoal ( Mt_Pessoal, Sx_Pessoal )
            VALUES  ( 1001      , 'M' );
-------------------------------------------------------------
ALTER TABLE Pessoal ALTER COLUMN Mt_Pessoal INTEGER NOT NULL;
```

Restrição DEFAULT

Prof. Me. Fred Lucena

DEFAULT

Define um valor padrão a ser utilizado pelo atributo.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Sx_Pessoal CHAR(1) DEFAULT('M') );
-------------------------------------------------------------------
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Sx_Pessoal CHAR(1) CONSTRAINT DF_Sx_Pessoal
                                  DEFAULT('M') );
```

DEFAULT

Define um valor padrão a ser utilizado pelo atributo.

```
Ex:
   INSERT INTO Pessoal ( Mt_Pessoal, Nm_Pessoal )
               VALUES  ( 1002, 'Jorge' );
   ------------------------------------------------------------
   SELECT * FROM Pessoal;
```

DEFAULT

Define um valor padrão a ser utilizado pelo atributo.

Ex:

```
ALTER TABLE Pessoal ADD DEFAULT('M') FOR Sx_Pessoal;
-----------------------------------------------------------
ALTER TABLE Pessoal ADD CONSTRAINT DF_Sx_Pessoal
                        DEFAULT('M') FOR Sx_Pessoal;
-----------------------------------------------------------
ALTER TABLE Pessoal DROP CONSTRAINT DF_Sx_Pessoal;
```

Restrição PRIMARY KEY

Prof. Me. Fred Lucena

PRIMARY KEY

Define o atributo como chave primária.

Ex:

```
CREATE TABLE Cargo ( Cd_Cargo INTEGER     PRIMARY KEY,
                     Ds_Cargo VARCHAR(50) NOT NULL );
-------------------------------------------------------------------
CREATE TABLE Cargo ( Cd_Cargo INTEGER CONSTRAINT PK_Cargo PRIMARY KEY,
                     Ds_Cargo VARCHAR(50) NOT NULL );
```

PRIMARY KEY

Define o atributo como chave primária.

Ex:

```
CREATE TABLE Cargo ( Cd_Cargo INTEGER     NOT NULL,
                     Ds_Cargo VARCHAR(50) NOT NULL,
                     PRIMARY KEY (Cd_Cargo) );
-------------------------------------------------------------
CREATE TABLE Cargo ( Cd_Cargo INTEGER     NOT NULL,
                     Ds_Cargo VARCHAR(50) NOT NULL,
                     CONSTRAINT PK_Cargo  PRIMARY KEY (Cd_Cargo) );
```

PRIMARY KEY

Define o atributo como chave primária.

Ex:

```
ALTER TABLE Cargo ADD PRIMARY KEY (Cd_Cargo) );
-----------------------------------------------------------
ALTER TABLE Cargo ADD CONSTRAINT PK_Cargo  PRIMARY KEY (Cd_Cargo);
```

Restrição FOREIGN KEY

Prof. Me. Fred Lucena

FOREIGN KEY

Define o atributo como chave estrangeira.

```
Ex:
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Cd_Cargo   INTEGER
                                  REFERENCES Cargo(Cd_Cargo) );
-------------------------------------------------------------------
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Cd_Cargo   INTEGER CONSTRAINT FK_Cargo
                                  REFERENCES Cargo(Cd_Cargo) );
```

FOREIGN KEY

Define o atributo como chave estrangeira.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Cd_Cargo   INTEGER,
                       FOREIGN KEY(Cd_Cargo)
                       REFERENCES Cargo(Cd_Cargo) );
```

FOREIGN KEY

Define o atributo como chave estrangeira.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Cd_Cargo   INTEGER,
                       CONSTRAINT FK_Cargo FOREIGN KEY(Cd_Cargo)
                       REFERENCES Cargo(Cd_Cargo) );
```

Restrição FOREIGN KEY (Ações em Cascata)

Exclusão em Cascata

FOREIGN KEY (Exclusão em Cascata)

```
ON DELETE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

NO ACTION

Não permite a exclusão de um registro que possua ligação com outra tabela. Uma mensagem de erro é apresentada e a exclusão não é efetivada.

NO ACTION é o padrão para o ON DELETE. Poderá, ou não, ser declarada.

FOREIGN KEY (Exclusão em Cascata)

```
ON DELETE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

CASCADE

Caso um registro possua ligação com outras tabelas, através de uma chave referenciada por chaves estrangeiras, todas os registros que contenham essas chaves estrangeiras serão excluídos.

FOREIGN KEY (Exclusão em Cascata)

```
ON DELETE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

SET NULL

Caso um registro possua ligação com outras tabelas, através de uma chave referenciada por chaves estrangeiras, todas as colunas que constituem a chave estrangeiras serão definidas como NULL.

FOREIGN KEY (Exclusão em Cascata)

```
ON DELETE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

SET DEFAULT

Caso um registro possua ligação com outras tabelas, através de uma chave referenciada por chaves estrangeiras, todas as colunas que constituem a chave estrangeiras serão definidas com os seus valores default.

FOREIGN KEY (Exclusão em Cascata)

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Cd_Cargo   INTEGER REFERENCES Cargo(Cd_Cargo)
                       ON DELETE  <NO ACTION|CASCADE|SET NULL|SET DEFAULT>  );
```

Restrição FOREIGN KEY (Ações em Cascata)

Alteração em Cascata

FOREIGN KEY (Alteração em Cascata)

```
ON UPDATE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

NO ACTION

Não permite a alteração de um registro que possua ligação com outra tabela. Uma mensagem de erro é apresentada e a exclusão não é efetivada.

NO ACTION é o padrão para o ON UPDATE. Poderá, ou não, ser declarada.

FOREIGN KEY (Alteração em Cascata)

```
ON UPDATE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

CASCADE

Caso um registro possua ligação com outras tabelas, através de uma chave referenciada por chaves estrangeiras, todas os registros que contenham essas chaves estrangeiras serão alterados.

FOREIGN KEY (Alteração em Cascata)

```
ON UPDATE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

SET NULL

Caso um registro possua ligação com outras tabelas, através de uma chave referenciada por chaves estrangeiras, todas as colunas que constituem a chave estrangeiras serão definidas como NULL.

FOREIGN KEY (Alteração em Cascata)

```
ON UPDATE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}
```

SET DEFAULT

Caso um registro possua ligação com outras tabelas, através de uma chave referenciada por chaves estrangeiras, todas as colunas que constituem a chave estrangeiras serão definidas com os seus valores default.

FOREIGN KEY (Alteração em Cascata)

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Cd_Cargo   INTEGER REFERENCES Cargo(Cd_Cargo)
                       ON UPDATE  <NO ACTION|CASCADE|SET NULL|SET DEFAULT>  );
```

Restrição IDENTITY

IDENTITY

Gera uma numeração de forma automática em um atributo.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER IDENTITY(1000,1),
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Sx_Pessoal CHAR(1) );
-------------------------------------------------------------
INSERT INTO Pessoal ( Nm_Pessoal, Sx_Pessoal )
            VALUES  ( 'Fred'    , 'M' );
-------------------------------------------------------------
@@IDENTITY -> Mostra o ultimo número gerado pelo IDENTITY;
TRUNCATE TABLE Pessoal; -> Apaga todos os registros e zera contador
```

IDENTITY

Interferindo no contador.

Ex:

```
SET IDENTITY_INSERT Pessoal ON;

INSERT INTO Pessoal ( Mt_Pessoal, Nm_Pessoal, Sx_Pessoal )
            VALUES  ( 1010, 'Mario', 'M' );

SELECT * FROM Pessoal;

SET IDENTITY_INSERT Pessoal OFF;
```

SEQUENCE

SEQUENCE

Gera uma numeração de forma automática.

Ex:

```
CREATE SEQUENCE Ct_Pessoa

AS INT

START WITH 1

INCREMENT BY 1;
```

SEQUENCE

Gera uma numeração de forma automática.

```
Ex:
   CREATE TABLE x ( Cd_Pessoa INTEGER PRIMARY KEY,
                    Nm_Pessoa VARCHAR(50) NOT NULL );

   INSERT INTO x ( Cd_Pessoa, Nm_Pessoa )
   VALUES ( NEXT VALUE FOR Ct_Pessoa, 'Fred' );

   DROP SEQUENCE Ct_Pessoa;
```

Restrição CHECK

CHECK

Valida o dado fornecido ao atributo.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER     PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL
                       Sx_Pessoal CHAR(1) CONSTRAINT CK_Sx_Pessoal
                       CHECK(Sx_Pessoal='M' OR Sx_Pessoal='F') );
```

CHECK

Valida o dado fornecido ao atributo.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER     PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL,
                       Sx_Pessoal CHAR(1) COLLATE Latin1_General_CS_AS
                       CHECK(Sx_Pessoal='M' OR Sx_Pessoal='F') );
```

COLLATE

`COLLATE` especifica a **collation**, que é um conjunto de regras de comparação de caracteres.

Ela determina:

1. **Sensibilidade a maiúsculas/minúsculas**
    1. `CS` → *Case Sensitive* (distingue maiúsculas/minúsculas)
    2. `CI` → *Case Insensitive* (ignora maiúsculas/minúsculas)
2. **Sensibilidade a acentos**
    1. `AS` → *Accent Sensitive* (distingue acentos)
    2. `AI` → *Accent Insensitive* (ignora acentos)
3. **Ordem de classificação (idioma/local)**
    1. Ex: `Latin1_General`, `SQL_Latin1_General`, `Modern_Spanish`, `Brazilian`, etc.

CHECK

Valida o dado fornecido ao atributo.

Ex:

```
INSERT INTO Pessoal VALUES ( 1003, 'Carlos', 'X' );

ALTER TABLE Pessoal
ADD CONSTRAINT CK_Sx_Pessoal
CHECK (Sx_Pessoal='M' OR Sx_Pessoal='F');


ALTER TABLE pessoal DROP CONSTRAINT CK_Sx_Pessoal;
```

Restrição UNIQUE

UNIQUE

Garante que um atributo não receba valores duplicados.

Ex:

```
CREATE TABLE Pessoal ( Mt_Pessoal INTEGER     PRIMARY KEY,
                       Nm_Pessoal VARCHAR(50) NOT NULL ,
                       Email      VARCHAR(50) NOT NULL
                       CONSTRAINT U_Email     UNIQUE );

ALTER TABLE Pessoal ADD CONSTRAINT U_Email UNIQUE (Email);
```

UNIQUE

Garante que um atributo não receba valores duplicados.

Ex:

```
INSERT INTO Pessoal VALUES ( 1005, 'Carlos', 'carlos@gmail.com' );

INSERT INTO Pessoal VALUES ( 1006, 'Maria' , 'carlos@gmail.com' );
```

Exercício: Crie as tabelas abaixo:

EMPREGADO		
CAMPO	**TIPO**	**RESTRIÇÃO**
Mt_Empregado	INT	AUTO INCREMENTO (1000,1) CHAVE PRIMÁRIA
Nome	VARCHAR(50)	NÃO NULO
Sexo	CHAR(1)	M ou F

DEPENDENTE		
CAMPO	**TIPO**	**RESTRIÇÃO**
Cd_Dependente	INT	AUTO INCREMENTO (100,1) CHAVE PRIMÁRIA
Nome	VARCHAR(50)	NÃO NULO
Sexo	CHAR(1)	M ou F
Mt_Empregado	INT	CHAVE ESTRANGEIRA

```sql
CREATE TABLE Empregado(
    Mt_Empregado   INTEGER      IDENTITY(1000,1) PRIMARY KEY,
    Nome           VARCHAR(50)  NOT NULL,
    Sexo           CHAR(1)      NOT NULL         CHECK(Sexo IN ('M','F'))
);
```

```sql
CREATE TABLE Dependente(
    Cd_Dependente  INTEGER      IDENTITY(100,1)  PRIMARY KEY,
    Nome           VARCHAR(50)  NOT NULL,
    Sexo           CHAR(1)      NOT NULL         CHECK(Sexo IN ('M','F')),
    Mt_Empregado   INTEGER      NOT NULL         REFERENCES Empregado(Mt_Empregado)
                                                 ON DELETE CASCADE
);
```

```sql
SELECT * FROM Empregado;

SELECT * FROM Dependente;
```

```
INSERT INTO Empregado  (Nome, Sexo)                VALUES ('Pedro' , 'M');
INSERT INTO Dependente (Nome, Sexo, Mt_Empregado) VALUES ('João'  , 'M', 1000),
                                                         ('Maria' , 'F', 1000);


INSERT INTO Empregado  (Nome, Sexo)                VALUES ('Ana'   , 'F');
INSERT INTO Dependente (Nome, Sexo, Mt_Empregado) VALUES ('Joana' , 'F', 1001),

                                                         ('Carlos', 'M', 1001);
SELECT * FROM Empregado;
SELECT * FROM Dependente;
```

```
DELETE FROM Empregado WHERE Mt_Empregado=1000;

SELECT * FROM Empregado;
SELECT * FROM Dependente;
```

Comandos DQL

Prof. Me. Fred Lucena

Preparando base de dados para exemplos

```
USE master;
IF DB_ID('Exercicios') IS NOT NULL
   DROP DATABASE Exercicios;
CREATE DATABASE Exercicios;
GO
USE Exercicios;
CREATE TABLE Agenda( Matricula   INTEGER      NOT NULL,
                     Nome        VARCHAR(50)  NOT NULL,
                     Nascimento  DATE         NOT NULL,
                     Sexo        CHAR(1)      NOT NULL,
                     Profissao   VARCHAR(20)  NOT NULL,
                     Salario     DECIMAL(10,2)        );
```

Preparando base de dados para exemplos

```
INSERT INTO Agenda
       ( Matricula, Nome, Nascimento, Sexo, Profissao, Salario )

VALUES ( 1,'Maria Jose da Silva'    ,'10/02/1980','F','Medico'    ,1235.68 ),
       ( 2,'João Paulo dos Santos'  ,'15/06/1982','M','Advogado'  ,1546.50 ),
       ( 3,'Ana Silva das Neves'    ,'05/08/1985','F','Engenheiro',2350.25 ),
       ( 4,'Silvana Maria Gonçalves','01/10/1988','F','Medico'    ,NULL    ),
       ( 5,'Ana Marisilva do Rego'  ,'11/11/1983','F','Advogado'  ,1500.00 ),
       ( 6,'Silva Santos Guedes'    ,'02/03/1984','M','Engenheiro',1620.00 ),
       ( 7,'Carlos José dos Santos' ,'06/04/1986','M','Medico'    ,NULL    ),
       ( 8,'Pedro Paulo Marques'    ,'17/06/1986','M','Advogado'  ,1600.00 ),
       ( 9,'Antonio Carlos da Silva','08/06/1986','M','Engenheiro',1930.00 );
```

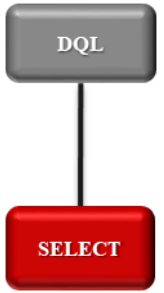
https://pastebin.com/tdbs4iPt

O que são os comandos DQL?

São os comandos destinados a consultar dados em tabelas.

A categoria DQL se resume ao comando:

SELECT



Sintaxe do comando SELECT:

SELECT <[tipo] [campos]> **FROM** <tabela/view> [<condição/ordem>];

[tipo] - parâmetro opcional que determina o tipo de registro (DISTINCT ou *)
[campos] - colunas da consulta
<tabela> - Nome das tabela a ser consultada
<condição> - Permite o estabelecimento dos critérios a seguir:
WHERE - Permite criar condições para filtrar dados utilizando operadores dos tipos:
- Relacionais - (<, <=, >, >=. =, <>);
- Lógicos - ([NOT] IN, [NOT] BETWEEN, [NOT] LIKE <%dado%>):
GROUP BY - Agrupa dados em uma consulta.
HAVING - Limita o resultado de uma consulta agrupada com GROUP BY.
ORDER BY - Especifica a coluna ou as colunas que serão ordenadas na consulta e a forma de ordenação (ascendente <ASC> ou descendente <DESC>).

Consultando dados:

```
SELECT * FROM Agenda;
```

Selecionando todas as colunas * de uma tabela.

```
SELECT Nome, Profissao
FROM Agenda;
```

Selecionando os dados da colunas específicas de uma tabela.

```
SELECT DISTINCT Profissao
FROM Agenda;
```

Selecionando os dados da coluna1 de uma tabela, sem repetição.

```
SELECT Nome AS 'NOME COMPLETO', Profissao AS PROFISSÃO
FROM Agenda;
```

Selecionando e nomeando colunas de uma tabela.

Comandos DQL - Cláusula WHERE

Prof. Me. Fred Lucena

Cláusula (WHERE)

Tem a função de estabelecer um predicado, condição para filtrar dados através da utilização de operadores lógicos, em uma consulta SQL.

Ex:

```
SELECT * FROM Tabela WHERE condição;
```

Cláusula (WHERE)

Operadores Relacionais	CONSULTA EM ÁLGEBRA RELACIONAL	CONSULTA SQL
<	$\sigma_{salario < 2000}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario < 2000;
<=	$\sigma_{salario <= 2000}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario <= 2000;
>	$\sigma_{salario > 2000}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario > 2000;
>=	$\sigma_{salario >= 2000}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario >= 2000;
=	$\sigma_{salario = 2000}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario = 2000;
<>	$\sigma_{salario <> 2000}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario <> 2000;

Cláusula (WHERE)

Operadores Lógicos	CONSULTA EM ÁLGEBRA RELACIONAL	CONSULTA SQL
IN	$\sigma_{\text{matricula IN (1,3,5,7)}}$ (**Agenda**)	SELECT * FROM Agenda WHERE matricula IN (1,3,5,7);
BETWEEN	$\sigma_{\text{salario BETWEEN 1500 AND 2000}}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario BETWEEN 1500 AND 2000;
LIKE (**%, _**)	$\sigma_{\text{nome LIKE '\%silva\%'}}$ (**Agenda**)	SELECT * FROM Agenda WHERE nome LIKE '%silva%';
IS NULL	$\sigma_{\text{salario IS NULL}}$ (**Agenda**)	SELECT * FROM Agenda WHERE salario IS NULL;
NOT	Nega o predicado nas consultas.	

Comandos DQL - Cláusula GROUP BY

Prof. Me. Fred Lucena

Cláusula (GROUP BY)

Tem a função de agrupar dados semelhantes em uma consulta SQL.

Ex:

```
SELECT * FROM Tabela GROUP BY coluna;
```

Cláusula (GROUP BY)

Ex: Conte a quantidade de pessoas cadastradas, por sexo, na tabela Agenda.

```
SELECT Sexo, COUNT(*) AS 'Qtd.'
FROM Agenda
GROUP BY Sexo;
```

Comandos DQL - Cláusula HAVING

Prof. Me. Fred Lucena

Cláusula (HAVING)

Tem função semelhante a cláusula WHERE, porém sua aplicação se restringe a consulta agrupadas (GROUP BY).

Ex:

```
SELECT * FROM Tabela GROUP BY coluna HAVING predicado;
```

Cláusula (HAVING)

Ex: Conte a quantidade de pessoas cadastradas, por profissão, na tabela Agenda, mostrando apenas as do sexo masculino.

```
SELECT  Profissao, Count(*)  AS  'Qtd.'
FROM  Agenda
GROUP BY  Profissao, Sexo
HAVING  Sexo = 'M';
```

Obs. A cláusula HAVING funciona como a cláusula WHERE em tabelas agrupadas.

Cláusula (HAVING)

Ex: Conte a quantidade de pessoas cadastradas, por profissão, na tabela Agenda, mostrando apenas as do sexo masculino.

```
SELECT  Profissao, Count(*)  AS  'Qtd.'
FROM  Agenda
WHERE  Sexo = 'M'
GROUP BY  Profissao;
```

Obs. A cláusula HAVING funciona como a cláusula WHERE em tabelas agrupadas.

Comandos DQL - Cláusula ORDER BY

Prof. Me. Fred Lucena

Cláusula (ORDER BY)

Tem função de ordenar dados em uma consulta SQL. Pode utilizar o complemento ASC (para ordenação ascendente) e DESC (para ordenação descendente).

Ex:

```
SELECT * FROM Tabela ORDER BY coluna ASC/DESC;
```

Cláusula (ORDER BY)

Ex: Selecione as colunas **Profissao** e **Nome**, na tabela **Agenda**, ordenando **Profissao** em ordem ascendente e **Nome** em ordem descendente.

```
SELECT  Profissao, Nome
FROM    Agenda
ORDER BY Profissao,  Nome  DESC;
```

Obs. A cláusula ASC pode ser omitida no caso de ordenação ascendente.

Comandos DQL - Cláusula TOP

Prof. Me. Fred Lucena

Cláusula (TOP(n))

Tem função de selecionar as **n** primeiras tuplas de uma consulta.

Ex:

```
SELECT TOP(n) * FROM Tabela;
```

Cláusula (TOP(n))

Ex: Mostre todos os dados dos três maiores salários na tabela Agenda.

```
SELECT  TOP (3)  salario
FROM  Agenda
ORDER BY  Salario  DESC;
```

Álgebra Relacional

Prof. Me. Fred Lucena

O que é Álgebra Relacional?

É uma linguagem formal (com formato estabelecido) e procedimental (as operações seguem uma ordem bem definida) com o objetivo de executar cálculos em relações. Suas operações são oriundas da teoria dos conjuntos.

A álgebra relacional ganhou especial atenção a partir da publicação do modelo relacional de dados onde, em 1970, o matemático britânico Edgar Frank Codd propôs a álgebra relacional como uma base para linguagens de consulta em banco de dados.

O que é Álgebra Relacional?

Definida por Frank Codd, é uma linguagem de consulta procedimental que consiste em um conjunto de operações objetivando produzir uma nova relação a partir de uma ou mais existentes. Na terminologia formal de modelo relacional:

- Uma linha é chamada de **tupla**;
- O cabeçalho da coluna é chamado de **atributo**;
- Tabela é chamada de **relação**;
- Tipos de dados permitidos em cada coluna é chamado de **domínio**.

Seleção (σ)

É uma operação indicada pela letra grega sigma (σ) que visa a obtenção de dados que satisfaçam determinada condição ou predicado.

$$\sigma_{\text{predicado}}\,(\,\textbf{Relação}\,)$$

predicado - condição que será aplicada a seleção.
relação – relação ou tabela de onde serão obtidos os dados.

Seleção (σ)

Ex: Selecione todas as pessoas com sexo feminino na tabela Agenda.

$$\sigma_{sexo='F'}(\text{Agenda})$$

```
SELECT * FROM Agenda WHERE Sexo = 'F';
```

Projeção (π)

É uma operação indicada pela letra grega pi (π) que visa a obtenção de dados referentes a algumas colunas de uma relação.

$$\pi_{\text{coluna1, coluna2}} \text{ (Relação)}$$

coluna1, coluna2 – colunas a serem mostradas.
relação – relação ou tabela de onde serão obtidos os dados.

Projeção (π)

Ex: Selecione todos os dados referentes as colunas Matricula e Nome da tabela Agenda.

$$\pi_{\text{Matricula, Nome}} \left(\textbf{Agenda} \right)$$

```
SELECT Matricula, Nome FROM Agenda;
```

Seleção (σ) e Projeção (π)

Ex: Selecione todos os dados das colunas Matricula e Nome na tabela Agenda, apenas dos empregados que possuem sexo = **'F'**.

$$\pi_{\text{Matricula, Nome}} (\sigma_{\text{sexo = 'F'}} (\textbf{ Agenda }))$$

SELECT Matricula, Nome **FROM** Agenda **WHERE** sexo = 'F';

Ordenação (τ)

É uma operação indicada pela letra grega tau (τ) que visa a ordenação dos dados consultados em uma relação.

$$\tau_{\text{coluna ord}} \, (\, \textbf{Relação} \,)$$

coluna – coluna a ser classificada.
ord – ordem de classificação -> asc (Ascendente), desc (Descendente)
relação – relação ou tabela de onde serão obtidos os dados.

Ordenação (τ)

Ex: Selecione todos os dados referentes as colunas Matricula e Nome, da tabela Agenda, ordenando a coluna nome na ascendente.

$$\tau_{\text{Nome [asc]}} \left(\pi_{\text{Matricula, Nome}} \left(\text{Agenda} \right) \right)$$

```
SELECT Matricula, Nome FROM Agenda ORDER BY Nome ASC;
```

Operações da Álgebra Relacional

Prof. Me. Fred Lucena

O que é Álgebra Relacional?

Na álgebra relacional existem sete operações básicas, são elas:

- Produto Cartesiano;
- União (Parcial e Total);
- Interseção;
- Diferença;
- Seleção;
- Projeção;
- Renomeação.

Criando Base para Testes

```
CREATE DATABASE Juncao;
GO
USE Juncao;
-------------------------------------------------
CREATE TABLE Cargo
            ( Cd_Cargo CHAR(3)     PRIMARY KEY,
              Cargo    VARCHAR(50) NOT NULL );
-------------------------------------------------
INSERT INTO Cargo
           ( Cd_Cargo, Cargo )
VALUES
           ( 'C01', 'Presidente' ),
           ( 'C02', 'Diretor' ),
           ( 'C03', 'Gerente' );
```

Criando Base para Testes

```
CREATE TABLE Empregado
            ( Matricula  CHAR(3)     PRIMARY KEY,
              Nome       VARCHAR(50) NOT NULL,
              Cd_Cargo   CHAR(3)     NOT NULL
              REFERENCES Cargo(Cd_Cargo) );
-------------------------------------------------
INSERT INTO Empregado
           ( Matricula, Nome, Cd_Cargo )
VALUES
           ( 'E01', 'Pedro','C01' ),
           ( 'E02', 'Paulo','C02' );
```

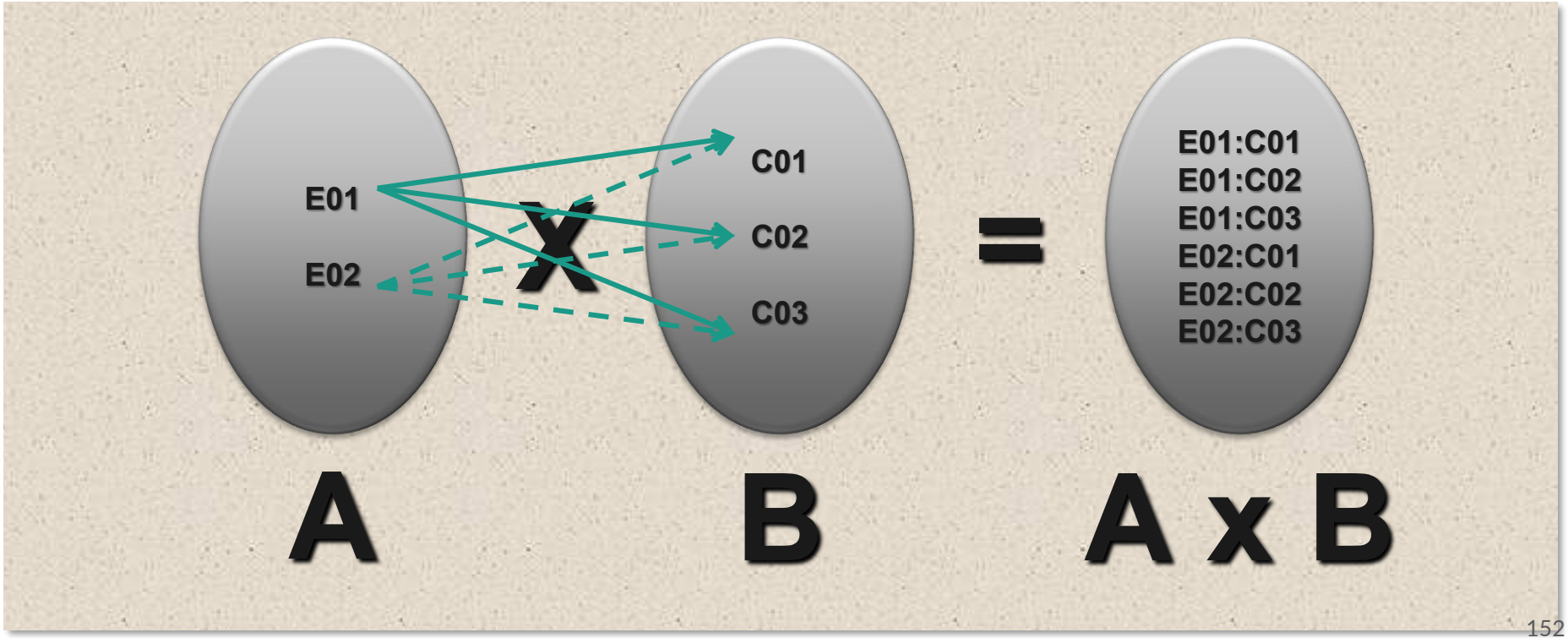
Criando Base para Testes

```
CREATE TABLE Empregado2
            ( Matricula  CHAR(3)     PRIMARY KEY,
              Nome       VARCHAR(50) NOT NULL,
              Cd_Cargo   CHAR(3)     NOT NULL
              REFERENCES Cargo(Cd_Cargo) );
-------------------------------------------------
INSERT INTO Empregado2
           ( Matricula, Nome, Cd_Cargo )
VALUES
           ( 'E02', 'Paulo'    ,'C02' ),
           ( 'E03', 'Don Diego','C01' ),
           ( 'E04', 'Zorro'    ,'C02' );
```

Produto Cartesiano

Prof. Me. Fred Lucena

Produto Cartesiano



Produto Cartesiano

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02

X

Cd_Cargo	Cargo
C01	Presidente
C02	Diretor
C03	Gerente

Matricula	Nome	Cd_Cargo	Cd_Cargo	Cargo
E01	Pedro	C01	C01	Presidente
E01	Pedro	C01	C02	Diretor
E01	Pedro	C01	C03	Gerente
E02	Paulo	C02	C01	Presidente
E02	Paulo	C02	C02	Diretor
E02	Paulo	C02	C03	Gerente

SELECT * FROM Empregado, Cargo;

Produto Cartesiano

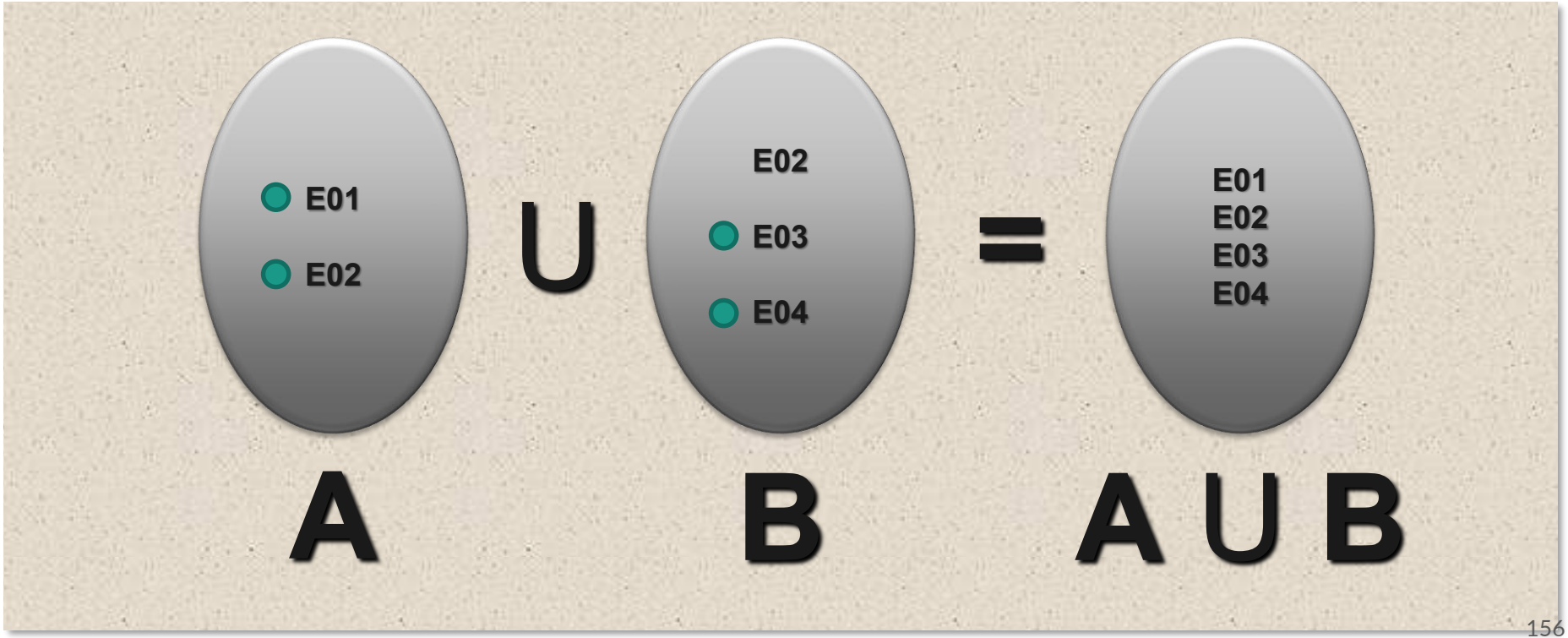
Matricula	Nome	Cd_Cargo	Cd_Cargo	Cargo
E01	Pedro	C01	C01	Presidente
E01	Pedro	C01	C02	Diretor
E01	Pedro	C01	C03	Gerente
E02	Paulo	C02	C01	Presidente
E02	Paulo	C02	C02	Diretor
E02	Paulo	C02	C03	Gerente

SELECT * FROM Empregado, Cargo;

União Parcial

Prof. Me. Fred Lucena

União Parcial



União Parcial

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02

U

Matricula	Nome	Cd_Cargo
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

SELECT * FROM Empregado UNION SELECT * FROM Empregado2;

União Parcial

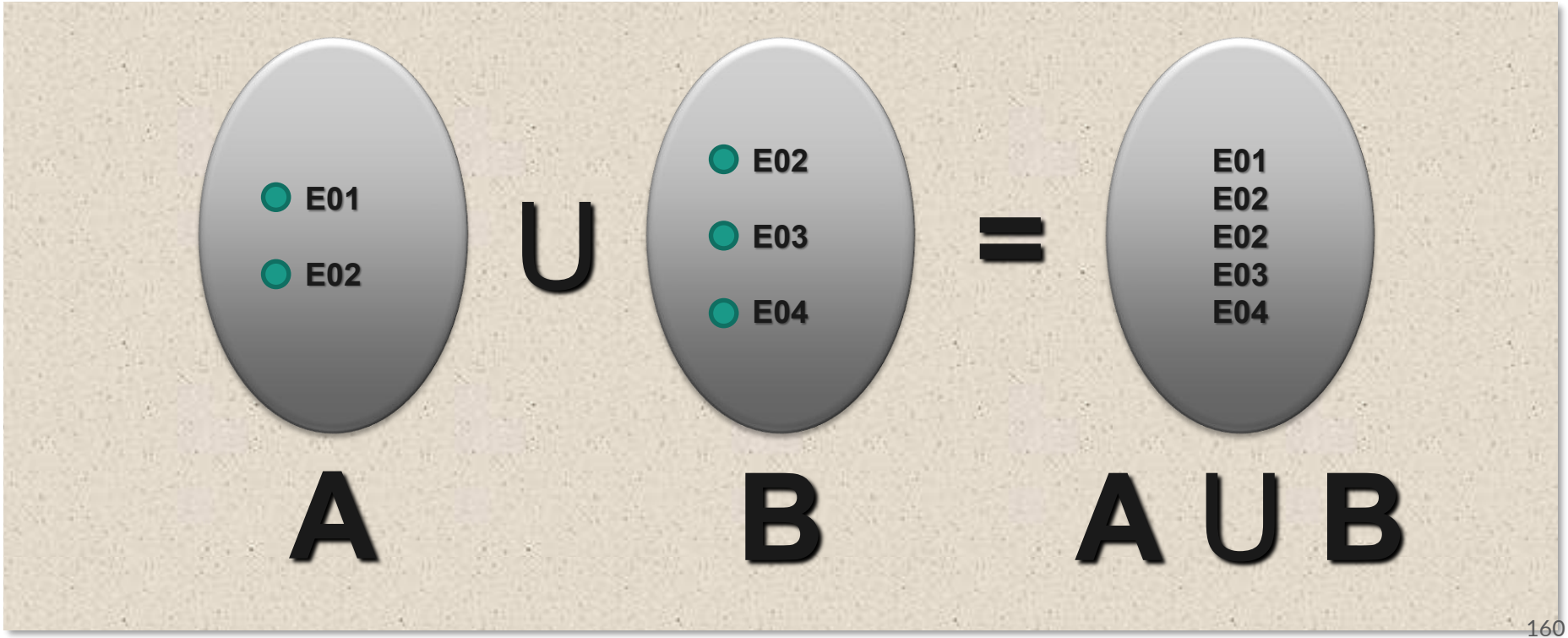
Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

SELECT * FROM Empregado UNION SELECT * FROM Empregado2;

União Total

Prof. Me. Fred Lucena

União Total



União Total

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02

U

Matricula	Nome	Cd_Cargo
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

SELECT * FROM Empregado UNION ALL SELECT * FROM Empregado2;

União Total

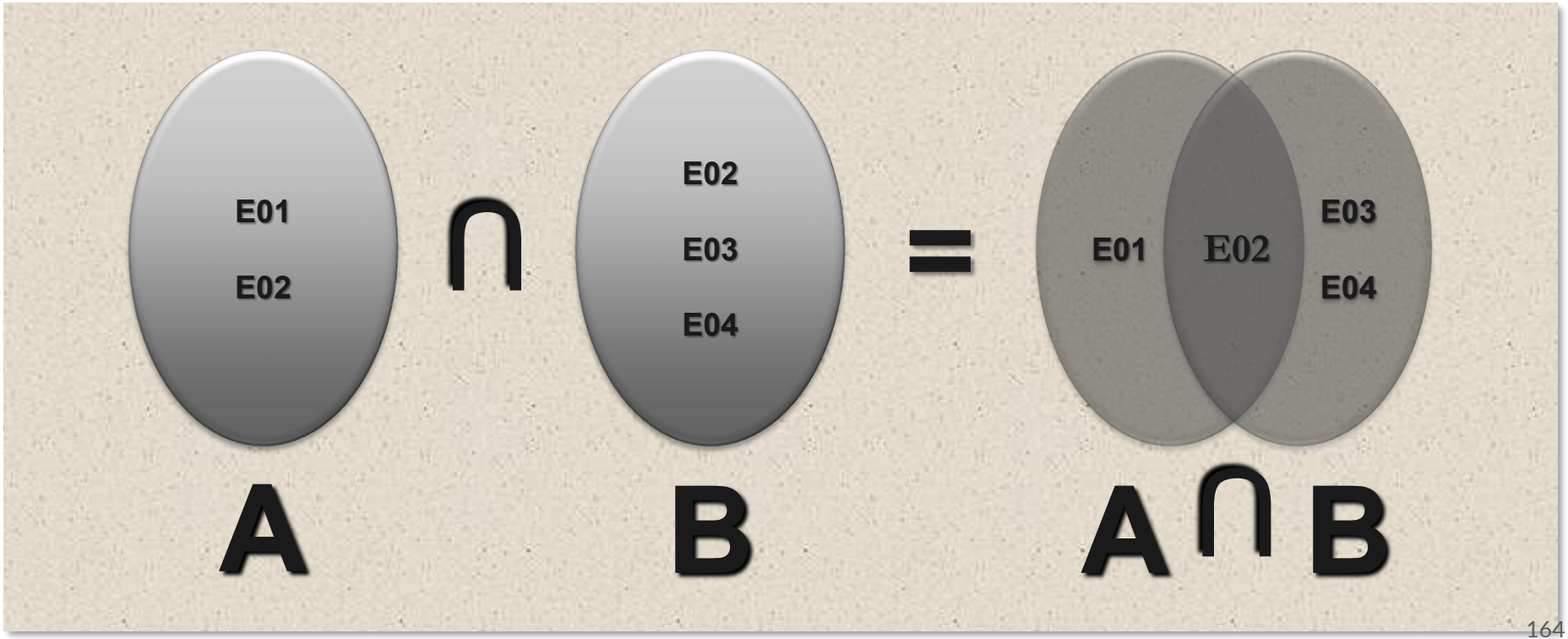
Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

SELECT * FROM Empregado UNION ALL SELECT * FROM Empregado2;

Interseção

Prof. Me. Fred Lucena

Interseção



Interseção

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02



Matricula	Nome	Cd_Cargo
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

Matricula	Nome	Cd_Cargo
E02	Paulo	C02

SELECT * FROM Empregado INTERSECT SELECT * FROM Empregado2;

Interseção

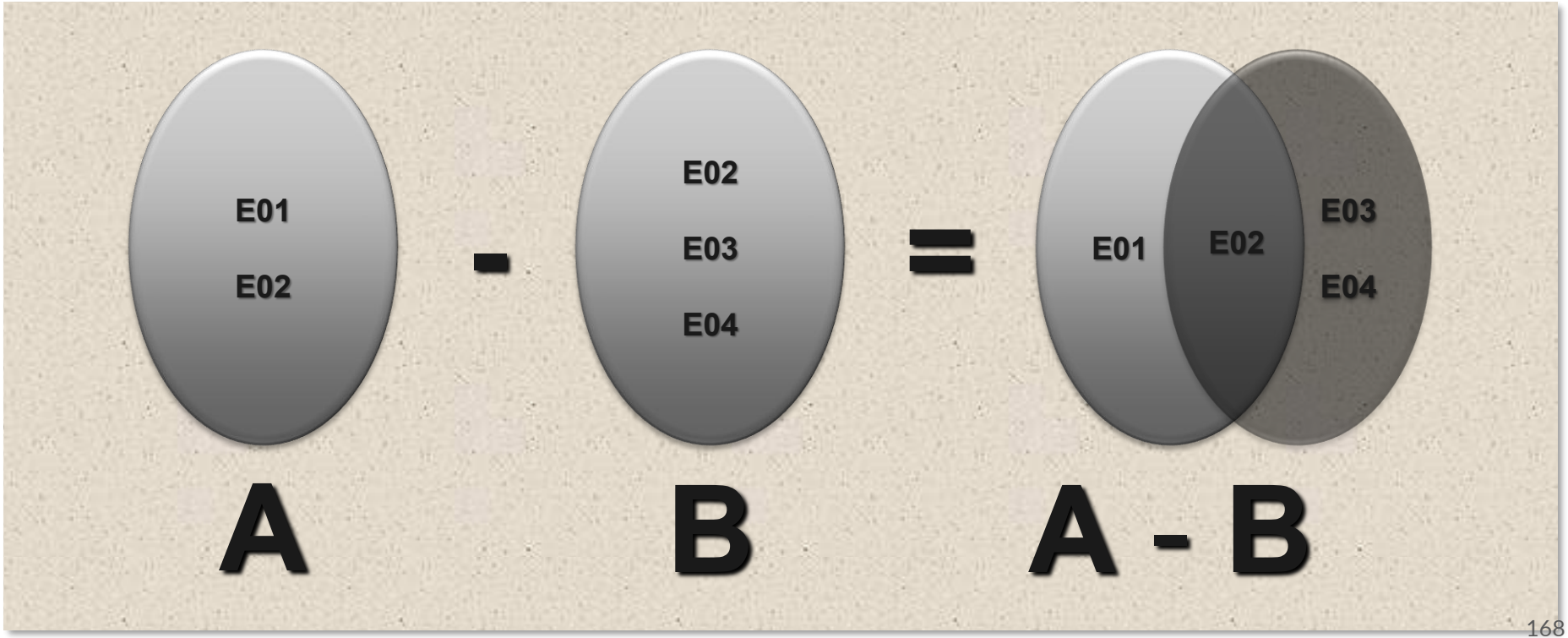
Matricula	Nome	Cd_Cargo
E02	Paulo	C02

SELECT * FROM Empregado INTERSECT SELECT * FROM Empregado2;

Diferença

Prof. Me. Fred Lucena

Diferença



Diferença

Matricula	Nome	Cd_Cargo
E01	Pedro	C01
E02	Paulo	C02

-

Matricula	Nome	Cd_Cargo
E02	Paulo	C02
E03	Don Diego	C01
E04	Zorro	C02

Matricula	Nome	Cd_Cargo
E01	Pedro	C01

SELECT * FROM Empregado EXCEPT SELECT * FROM Empregado2;

Diferença

Matricula	Nome	Cd_Cargo
E01	Pedro	C01

```
SELECT * FROM Empregado EXCEPT SELECT * FROM Empregado2;
```

Junção

Prof. Me. Fred Lucena

O que é Junção?

Segundo a língua Portuguesa:

s. f.

- 1. Ato ou efeito de juntar.
- 2. Ponto onde duas ou mais coisas se juntam.
- 3. Reunião.
- 4. Confluência.

Segundo a álgebra relacional:

Operação utilizada para combinar dados de duas ou mais relações (tabelas).

Junção

Estilo	Construção
ANSI 89	SELECT <Campos> FROM A, B WHERE A.Coluna = B.Coluna
ANSI 92	SELECT <Campos> FROM A INNER JOIN B ON A.Coluna = B.Coluna

Criando Base para Testes

```
CREATE TABLE marcas ( marca VARCHAR(50),
                      nome  VARCHAR(50) );

INSERT INTO marcas
            ( marca, nome)
VALUES
            ( 'VW'      ,'Volkswagem'      ),
            ( 'Ford'    ,'Ford'            ),
            ( 'GM'      ,'General Motors'  ),
            ( 'Fiat'    ,'Fiat'            ),
            ( 'Renault','Renault'          ),
            ( 'MB'      ,'Mercedes-Benz'   );
```

Criando Base para Testes

```
CREATE TABLE carros ( modelo VARCHAR (100),
                      marca  VARCHAR (50),
                      ano    INTEGER,
                      cor    VARCHAR (20));

INSERT INTO carros
            ( modelo, marca, ano, cor )
VALUES
            ( 'Fox'      ,'VW'       ,2005 ,'preto' ),
            ( 'Ecosport','Ford'     ,2009 ,'verde' ),
            ( 'KA'       ,'Ford'     ,2008 ,'prata' ),
            ( 'Punto'    ,'Fiat'     ,2008 ,'branco'),
            ( 'Uno'      ,'Fiat'     ,2007 ,'preto' ),
            ( 'Stilo'    ,'Fiat'     ,2004 ,'prata' ),
            ( 'Uno'      ,'Fiat'     ,2009 ,'branco'),
            ( '207'      ,'Peugeot'  ,2010 ,'prata' ),
            ( '300 C'    ,'Chrysler',2008 ,'verde' );
```

Tipos de Junção

Prof. Me. Fred Lucena

Tipos de Junção

As junções podem ser divididas em três categorias básicas:

- **INNER JOIN**
- **OUTER JOIN**
  - LEFT JOIN
  - RIGHT JOIN
  - FULL JOIN
- **CROSS JOIN**

[INNER] JOIN

O **INNER JOIN** junta todos os registros que possuam correspondência nas tabelas relacionadas.

EX:

```
SELECT marcas.nome, carros.modelo
FROM   marcas
INNER JOIN carros ON carros.marca = marcas.marca;
```

LEFT [OUTER] JOIN

O **LEFT** [OUTER] **JOIN** junta todos os registros da primeira tabela e na segunda, apenas aqueles que possuem relação.

EX:

```
SELECT m.nome, c.modelo
FROM   marcas AS m
LEFT JOIN carros AS c ON c.marca = m.marca;
```

RIGHT [OUTER] JOIN

O **RIGHT** [OUTER] **JOIN** junta todos os registros da segunda tabela e na primeira, apenas aqueles que possuem relação.

EX:

```
SELECT m.nome, c.modelo
FROM   marcas AS m
RIGHT JOIN  carros AS c ON c.marca = m.marca;
```

FULL [OUTER] JOIN

O **FULL** [OUTER] **JOIN** é o equivalente ao uso do **LEFT JOIN** e **RIGHT JOIN** juntos, ou seja, mostra todos os registros das duas tabelas, independente de existirem correspondências.

.

EX:

```
SELECT m.nome, c.modelo
FROM   marcas AS m
FULL JOIN carros AS c ON c.marca = m.marca;
```

CROSS JOIN

O **CROSS JOIN** junta todos os registros de duas tabelas formando um produto cartesiano. Este tipo de junção não permite o uso da cláusula **ON**.

.

EX:

```
SELECT m.nome, c.modelo
FROM   marcas AS m
CROSS JOIN carros AS c;
```

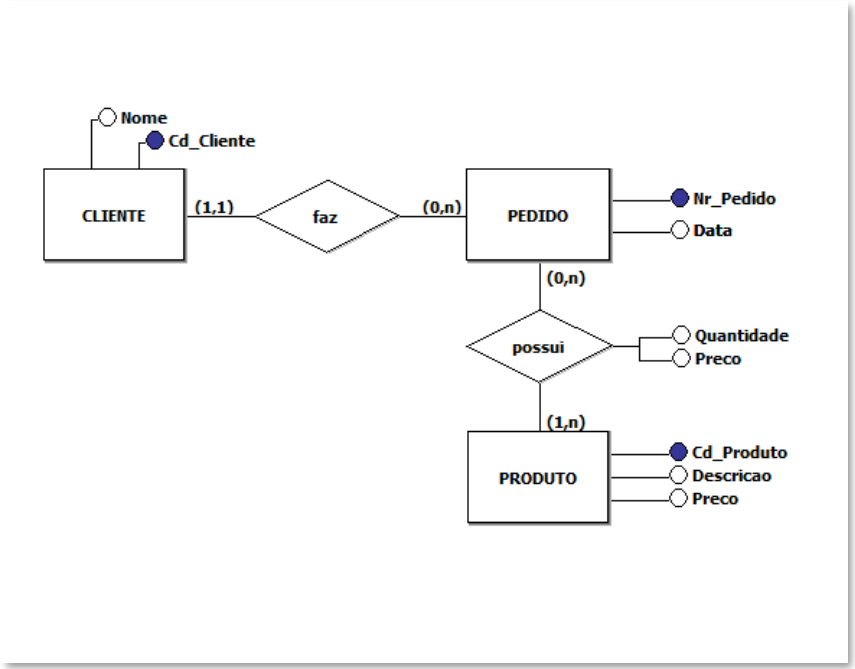
Exercício

Preparando a Base de Dados

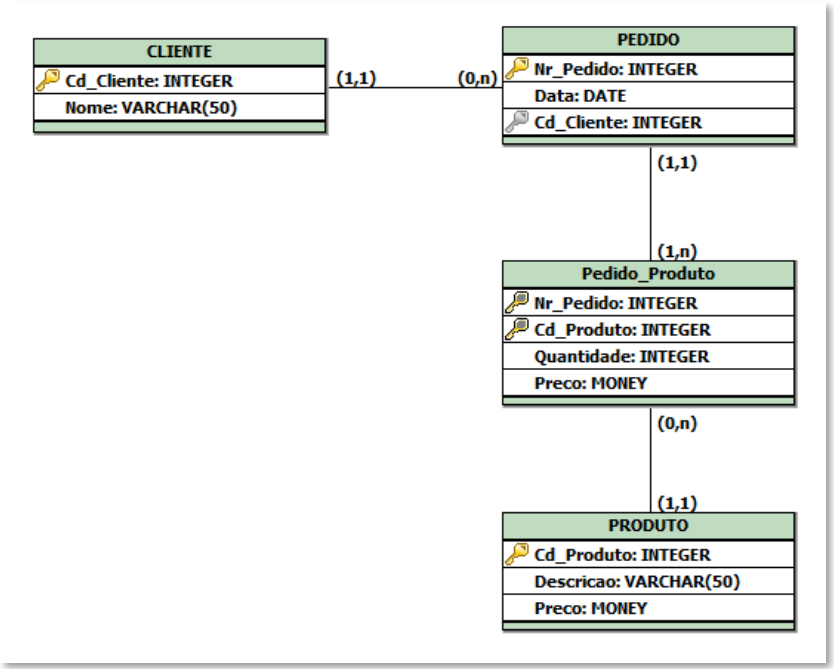
Exercícios

https://pastebin.com/c1HwJH4T

Modelo Conceitual



Modelo Lógico



Modelo Físico

```
USE master;
IF DB_ID('Pedido') IS NOT NULL
   DROP DATABASE Pedido;
CREATE DATABASE Pedido;
GO
USE Pedido;
-------------------------------------------------------------------------------
CREATE TABLE Cliente(
            Cd_Cliente INTEGER       IDENTITY(1000,1) PRIMARY KEY,
            Nome       VARCHAR(50)
);
-------------------------------------------------------------------------------
CREATE TABLE Pedido(
            Nr_Pedido  INTEGER       IDENTITY         PRIMARY KEY,
            Data       DATE          NOT NULL,
            Cd_Cliente INTEGER       NOT NULL         REFERENCES Cliente(Cd_Cliente)
);
-------------------------------------------------------------------------------
CREATE TABLE Produto(
            Cd_Produto INTEGER       IDENTITY         PRIMARY KEY,
            Descricao  VARCHAR(50)   NOT NULL,
            Preco      DECIMAL(10,2) NOT NULL
);
-------------------------------------------------------------------------------
CREATE TABLE Pedido_Produto(
            Nr_Pedido  INTEGER       NOT NULL         REFERENCES Pedido(Nr_Pedido),
            Cd_Produto INTEGER       NOT NULL         REFERENCES Produto(Cd_Produto),
            Quantidade INTEGER       NOT NULL,
            Preco      DECIMAL(10,2) NOT NULL,
            PRIMARY KEY(Nr_Pedido,Cd_Produto)
);
```

Modelo Físico

```
INSERT INTO cliente
            ( Nome )
VALUES
            ( 'PEDRO'  ),
            ( 'CARLOS' ),
            ( 'MARIA'  );
-----------------------------------------------------------
INSERT INTO pedido
            ( Data, Cd_Cliente )
VALUES
            ( '02/11/2010',1000 ),
            ( '05/11/2010',1001 ),
            ( '07/11/2010',1001 ),
            ( '08/11/2010',1002 );
-----------------------------------------------------------
INSERT INTO produto
            ( Descricao, Preco)
VALUES
            ( 'MOUSE'  ,180 ),
            ( 'TECLADO',120 ),
            ( 'MONITOR',550 );
-----------------------------------------------------------
INSERT INTO pedido_produto
            ( Nr_Pedido, Cd_Produto, Quantidade, Preco )
VALUES
            ( 1, 1, 10, 150 ),
            ( 1, 2, 15, 120 ),
            ( 1, 3, 20, 530 ),
            ( 2, 1, 5 , 140 ),
            ( 2, 2, 8 , 130 ),
            ( 3, 1, 3 , 150 ),
            ( 3, 2, 7 , 130 ),
            ( 3, 3, 11, 570 ),
            ( 4, 1, 13, 145 ),
            ( 4, 2, 17, 135 );
```

Exercícios

```
INSERT INTO cliente
            ( Nome )
VALUES
            ( 'PEDRO'  ),
            ( 'CARLOS' ),
            ( 'MARIA'  );
-------------------------------------------------------------
INSERT INTO pedido
            ( Data, Cd_Cliente )
VALUES
            ( '02/11/2010',1000 ),
            ( '05/11/2010',1001 ),
            ( '07/11/2010',1001 ),
            ( '08/11/2010',1002 );
```

Exercícios

```
INSERT INTO produto
            ( Descricao, Preco)
VALUES
            ( 'MOUSE'   ,180 ),
            ( 'TECLADO',120 ),
            ( 'MONITOR',550 );
```

Exercícios

```
INSERT INTO pedido_produto
            ( Nr_Pedido, Cd_Produto, Quantidade, Preco )
VALUES
            ( 1, 1, 10, 150 ),
            ( 1, 2, 15, 120 ),
            ( 1, 3, 20, 530 ),
            ( 2, 1, 5 , 140 ),
            ( 2, 2, 8 , 130 ),
            ( 3, 1, 3 , 150 ),
            ( 3, 2, 7 , 130 ),
            ( 3, 3, 11, 570 ),
            ( 4, 1, 13, 145 ),
            ( 4, 2, 17, 135 );
```

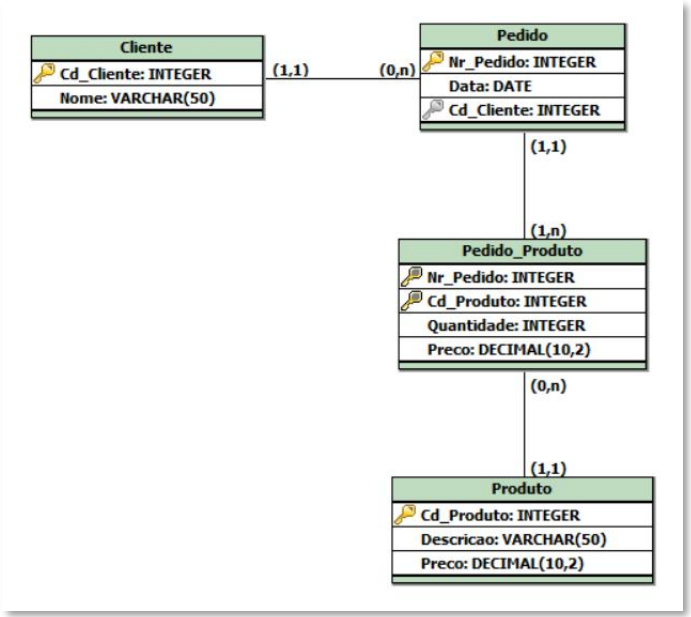
Exercício

Gerando as Consultas

Exercícios

Utilizando produto cartesiano, liste o código do cliente, o nome do cliente, o número do pedido e a data do pedido apenas dos clientes que possuem pedidos.

```
SELECT CLI.Cd_Cliente,
       CLI.Nome,
       PED.Nr_Pedido,
       PED.Data
FROM   CLIENTE AS CLI, PEDIDO AS PED
WHERE  CLI.Cd_Cliente = PED.Cd_Cliente;
```



Exercícios

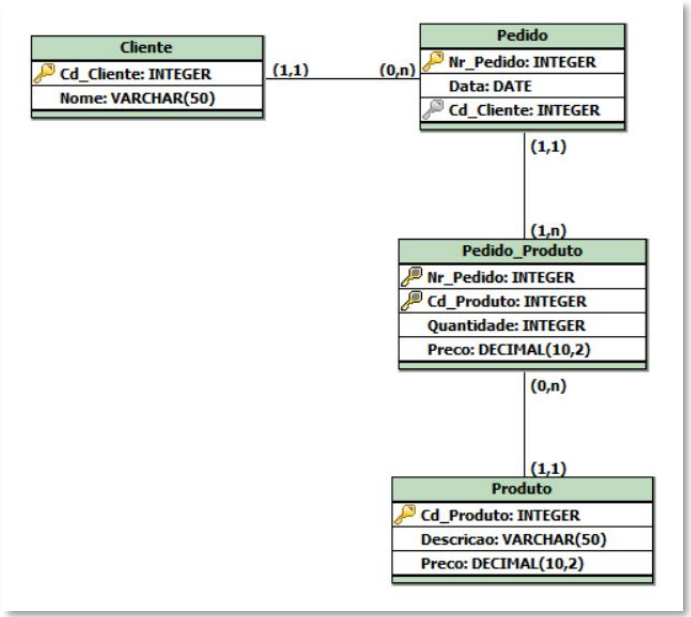
Utilizando INNER JOIN, liste o código do cliente, o nome do cliente, o número do pedido e a data do pedido apenas dos clientes que possuem pedidos.

```
SELECT CLI.Cd_Cliente,
       CLI.Nome,
       PED.Nr_Pedido,
       PED.Data
FROM   CLIENTE AS CLI
INNER  JOIN PEDIDO AS PED ON CLI.Cd_Cliente = PED.Cd_Cliente;
```

Exercícios

Liste, utilizando produto cartesiano, do Pedido (número e data), do Cliente (nome) e do Pedido_Produto (código do produto, quantidade e preço).

```
SELECT  Pedido.Nr_Pedido,
        CONVERT(char,Pedido.Data,103),
        Cliente.Nome,
        Pedido_Produto.Cd_Produto,
        Pedido_Produto.Quantidade,
        Pedido_Produto.Preco
FROM    Pedido,Cliente,Pedido_Produto
WHERE   Cliente.Cd_Cliente = Pedido.Cd_Cliente AND
        Pedido.Nr_Pedido   = Pedido_Produto.Nr_Pedido;
```



Exercícios

Liste, utilizando INNER JOIN, do Pedido (número e data), do Cliente (nome) e do Pedido_Produto (código do produto, quantidade e preço).

```
SELECT Pedido.Nr_Pedido,
       CONVERT(char,Pedido.Data,103),
       Cliente.Nome,
       Pedido_Produto.Cd_Produto,
       Pedido_Produto.Quantidade,
       Pedido_Produto.Preco
FROM   Pedido
JOIN   Cliente        ON Pedido.Cd_Cliente = Cliente.Cd_Cliente
JOIN   Pedido_Produto ON Pedido.Nr_Pedido  = Pedido_Produto.Nr_Pedido;
```

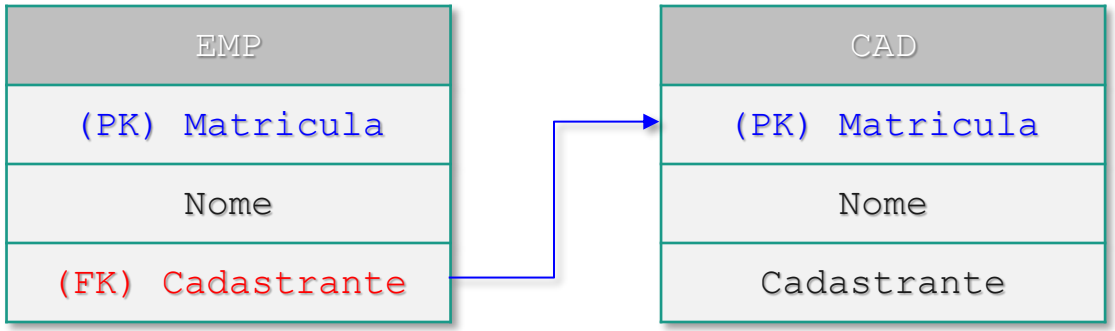
Exercícios

A partir da criação e alimentação da tabela abaixo, faça uma consulta que mostre, além da matrícula e nome do empregado cadastrado, o nome do cadastrante.

```
CREATE TABLE Empregado ( Matricula   INTEGER PRIMARY KEY,
                         Nome        VARCHAR(50),
                         Cadastrante INTEGER REFERENCES Empregado (Matricula) );
-------------------------------------------------------------------------------
INSERT INTO Empregado ( Matricula, Nome, Cadastrante)
              VALUES ( 1, 'PEDRO' ,1),
                     ( 2, 'PAULO' ,1),
                     ( 3, 'MAURO' ,1),
                     ( 4, 'MARIA' ,2),
                     ( 5, 'KARLA' ,2),
                     ( 6, 'LAURA' ,2);
```

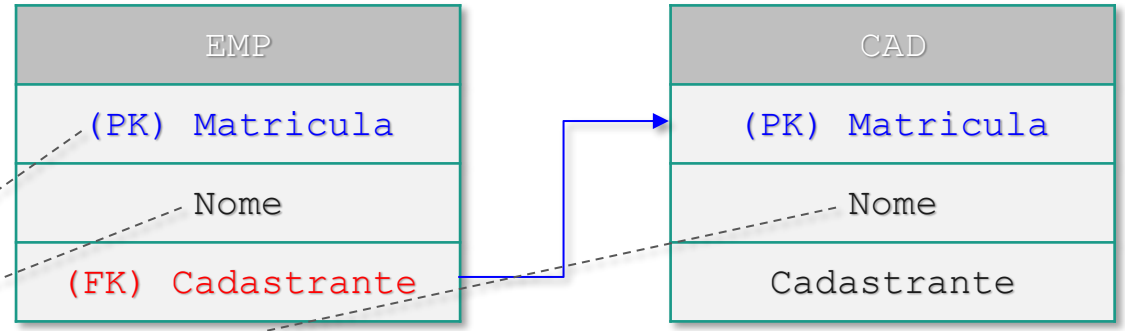
Exercícios

A partir da criação e alimentação da tabela abaixo, faça uma consulta que mostre, além da matrícula e nome do empregado cadastrado, o nome do cadastrante.



Exercícios

A partir da criação e alimentação da tabela abaixo, faça uma consulta que mostre, além da matrícula e nome do empregado cadastrado, o nome do cadastrante.




```
SELECT EMP.Matricula,
       EMP.Nome,
       CAD.Nome AS Cadastrante
FROM   Empregado AS EMP
JOIN   Empregado AS CAD ON EMP.Cadastrante = CAD.Matricula;
```

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
SISTEMAS PARA INTERNET
Linguagem SQL

Prof. Me. Fred Lucena

Triggers

Prof. Me. Fred Lucena

O que é um trigger?

É um bloco de comandos escritos na linguagem do SGBD (Transact-SQL) e que é automaticamente executado quando da utilização de um dos comandos **INSERT**, **UPDATE** ou **DELETE** em uma tabela do banco de dados.

Sintaxe de um trigger

```
CREATE OR ALTER  TRIGGER <nome_do_trigger>
ON
        <tabela>
FOR / AFTER
        INSERT,  UPDATE,  DELETE
AS
        <instruções>
```

Trigger (Exemplo 01)

https://pastebin.com/4jYtVNdM

```
CREATE OR ALTER TRIGGER Controle
ON
      Cadfun
FOR
      INSERT, UPDATE, DELETE
AS
      SELECT * FROM Cadfun;
```

Trigger (Exemplo 01 – Testando...)

```
INSERT INTO cadfun
VALUES (99,'Teste','99','Testador',9999.99);
-----------------------------------------------------
UPDATE cadfun
SET    NOME   = 'Novo Teste'
WHERE  codfun = '99';
-----------------------------------------------------
DELETE FROM cadfun
WHERE codfun = '99';
```

Listando os triggers existentes

```
SELECT *
FROM   sysobjects
WHERE  xtype = 'TR';
```

Desabilitando Triggers

Prof. Me. Fred Lucena

Desabilitando Triggers

```
DISABLE TRIGGER <nome> ON <tabela>;
```

Habilitando Triggers

Prof. Me. Fred Lucena

Habilitando Triggers

```
ENABLE TRIGGER <nome> ON <tabela>;
```

Excluindo Triggers

Prof. Me. Fred Lucena

Excluindo Triggers

```
DROP TRIGGER <nome>;
```

Trigger (Exemplo 02)

Desenvolva triggers que, durante as operações de inclusão, alteração e exclusão, emitam as seguintes mensagens:

- Registro(s) incluído(s) com sucesso!
- Registro(s) alterado(s) com sucesso!
- Registro(s) excluído(s) com sucesso!

Trigger (Exemplo 02 – Solução 1)

```
CREATE TRIGGER mensagemI
ON
    cadfun
FOR
    INSERT
AS
    PRINT 'Registro(s) incluído(s) com sucesso!';
```

Trigger (Exemplo 02 – Solução 2)

```
CREATE TRIGGER mensagemU
ON
    cadfun
FOR
    UPDATE
AS
    PRINT 'Registro(s) alterado(s) com sucesso!';
```

Trigger (Exemplo 02 – Solução 3)

```
CREATE TRIGGER mensagemD
ON
    cadfun
FOR
    DELETE
AS
    PRINT 'Registro(s) excluído(s) com sucesso!';
```

Triggers (Tabelas Temporárias)

Prof. Me. Fred Lucena

O que são as tabelas temporárias?

Ao executar as operações de inserção, alteração e exclusão, o SQL Server cria tabelas temporárias relativas ao tipo de operação. Essas tabelas só estão acessíveis durante o processo de execução do trigger.

As tabelas temporárias e a operação

Na inserção -> **INSERTED**;

Na Exclusão -> **DELETED**;

Na alteração -> **INSERTED** e **DELETED**.

Trigger (Exemplo 02 – Solução 4)

```
CREATE OR ALTER TRIGGER mensagem
ON
    cadfun
FOR
    INSERT, UPDATE, DELETE
AS
    IF  EXISTS(SELECT * FROM inserted)
        IF  EXISTS(SELECT * FROM deleted)
            PRINT 'Registro alterado com sucesso!'
        ELSE
            PRINT 'Registro incluido com sucesso!'
    ELSE
        PRINT 'Registro excluído com sucesso!';
```

Triggers (Exercícios)

Prof. Me. Fred Lucena

Trigger (Exercício 01)

Sendo a tabela “morto” uma cópia vazia de uma tabela orígem, crie um trigger capaz de copiar para a tabela morto, os registros excluídos na tabela original.

Trigger (Exercício 01 – Resposta)

```
CREATE OR ALTER TRIGGER controle
ON
    cadfun
FOR
    DELETE
AS
    INSERT INTO morto SELECT * FROM DELETED;
```

Trigger (Exercício 01 – Resposta)

```
DELETE FROM cadfun WHERE codfun = '11';

SELECT * FROM morto;

INSERT INTO cadfun SELECT * FROM morto;
```

Trigger Instead Of

Prof. Me. Fred Lucena

O que trigger instead of?

Este tipo de trigger não é apenas disparado por um INSERT, UPDATE ou DELETE, mas substitui estes comandos ficando a cargo do trigger a execução do procedimento.

Trigger Instead Of (Exemplo 01)

```
CREATE OR ALTER TRIGGER incluir
ON
   cadfun INSTEAD OF INSERT
AS
   PRINT 'Registro incluído com sucesso.';
```

```
INSERT INTO cadfun VALUES (99,'Teste','99','Testador',9999.99);
```

Trigger Instead Of (Exemplo 02)

```
CREATE OR ALTER TRIGGER incluir
ON
    cadfun INSTEAD OF INSERT
AS
    INSERT INTO cadfun SELECT * FROM INSERTED;
```

Trigger Instead Of (Exercícios)

Prof. Me. Fred Lucena

Trigger Instead Of (Exercício 01)

Crie um trigger que impeça a inclusão em cadfun de alguém ganhando menos de 1000 reais. O trigger deverá apresentar a mensagem de Inclusão não autorizada quando o salário for menor que 1000 ou inserir os dados em caso contrário.

Trigger (Exercício 01 – Resposta)

```
CREATE OR ALTER TRIGGER incluir
ON
   cadfun
INSTEAD OF INSERT
AS
   IF EXISTS(SELECT salario FROM INSERTED WHERE salario < 1000)
       PRINT 'Salário não autorizado! ';
   ELSE
       INSERT INTO cadfun SELECT * FROM INSERTED;
```

```
INSERT INTO cadfun VALUES (99,'Teste','99','Testador',900.00);
```

Stored Procedures

Prof. Me. Fred Lucena

O que é um stored procedure?

É um bloco de comandos escritos na linguagem do SGBD (Transact-SQL) com o objetivo de executar determinada tarefa. *Fica armazenado no Banco de Dados e permite executar várias tarefas previamente definidas.*

Sintaxe de um stored procedure

```
CREATE OR ALTER  PROCEDURE <nome_do_procedimento>
        [parâmetros]
AS
        <instruções>;
```

Stored Procedure (Exemplo 01)

```
CREATE OR ALTER PROCEDURE SP_Escrever
AS
        PRINT 'Olá Pernambuco';

-----------------------------------------
EXECUTE SP_Escrever;
```

Stored Procedures

(Utilizando Parâmetros)

Prof. Me. Fred Lucena

Stored Procedure (Utilizando Parâmetros...)

```
CREATE OR ALTER PROCEDURE SP_Escrever
      @Texto VARCHAR(35)
AS
      PRINT @Texto;

-----------------------------------------------------
EXECUTE SP_Escrever 'Olá Recife';
```

Stored Procedure (Fazendo Cálculos...)

```
CREATE OR ALTER PROCEDURE SP_Calcular
        @Valor1 INTEGER, @Valor2 INTEGER
AS
        PRINT 'Resultado = ' + STR(@Valor1 + @Valor2);

-------------------------------------------------------
EXECUTE SP_Calcular 10, 5;
```

Stored Procedure (Fazendo Cálculos...)

```
CREATE OR ALTER PROCEDURE SP_Calcular
      @Valor1 INTEGER, @Valor2 INTEGER
AS
      PRINT 'Resultado = ' + LTRIM(STR(@Valor1 + @Valor2));

-----------------------------------------------------
EXECUTE SP_Calcular 10, 5;
```

Stored Procedure (Fazendo Cálculos...)

```
CREATE OR ALTER PROCEDURE SP_Calcular
      @Valor1 INTEGER, @Valor2 INTEGER
AS
      PRINT 'Resultado = ' + CAST(@Valor1 + @Valor2 AS VARCHAR);

-------------------------------------------------------------
EXECUTE SP_Calcular 10, 5;
```

Stored Procedures

(Utilizando Parâmetros Opcionais)

Prof. Me. Fred Lucena

Stored Procedure (Utilizando Parâmetros Opcionais...)

```
CREATE OR ALTER PROCEDURE SP_ConsultarFuncao
      @funcao VARCHAR(35) = NULL
AS
      IF @funcao IS NULL
         SELECT * FROM CadFun;
      ELSE
         SELECT * FROM CadFun WHERE funcao = @funcao;

-------------------------------------------------------------
EXECUTE SP_ConsultarFuncao;  ou  EXECUTE SP_ConsultarFuncao 'VENDEDOR';
```

Stored Procedures

(Declarando Variáveis)

Prof. Me. Fred Lucena

Stored Procedure (Declarando Variáveis...)

```
CREATE OR ALTER PROCEDURE SP_ConsultarFuncao
AS
      DECLARE @funcao VARCHAR(35)
      SET @funcao = 'Vendedor'
      SELECT * FROM CadFun WHERE funcao = @funcao;

-------------------------------------------------------------
EXECUTE SP_ConsultarFuncao;
```

Stored Procedure (Consulta com Parâmetros...)

```
CREATE OR ALTER PROCEDURE sp_InserirDados
                          @codfun   INTEGER,
                          @nome     VARCHAR(40),
                          @depto    VARCHAR(2),
                          @funcao   VARCHAR(20),
                          @salario  DECIMAL(10,2)
AS
       INSERT INTO cadfun VALUES (@codfun,
                                  @nome,
                                  @depto,
                                  @funcao,
                                  @salario);
```

Stored Procedure (Consulta com Parâmetros...)

```
EXEC sp_InserirDados 99995, 'teste 3', '99', 'SP',999.99


EXEC sp_InserirDados
     @codfun   = 99999,
     @nome     = 'Teste 3',
     @depto    = '99’,
     @funcao   = 'SP’,
     @salario  = 999.99
```

Stored Procedure (Usando teste condicional...)

```
CREATE OR ALTER PROCEDURE SP_IF
        @nome VARCHAR(35), @sexo CHAR(1)
AS
        DECLARE @tratamento CHAR(5)
        IF @sexo = 'M'
           SET @tratamento = 'Sr. '
        ELSE
           SET @tratamento = 'Sra. '
        PRINT 'Prazer ' + @tratamento + @nome;
```

```
EXECUTE SP_IF 'Maria', 'F';
```

Stored Procedures

(Estrutura de Repetição WHILE)

Prof. Me. Fred Lucena

Stored Procedure (Estrutura de Repetição WHILE...)

```
WHILE (condição)
      BEGIN
            [instruções]
            [instruções]
            [BREAK]
      END;
```

Stored Procedure (Estrutura de Repetição WHILE...)

```
CREATE OR ALTER PROCEDURE SP_Contar
        @Limite INT
AS
        DECLARE @x INT;
        SET @x=1;
        WHILE (@x <= @Limite)
                BEGIN
                        PRINT @x;
                        SET @x += 1;
                END;
```

Stored Procedure (Estrutura de Repetição WHILE...)

```
CREATE OR ALTER PROCEDURE SP_Contar
        @Limite INT
AS
        DECLARE @x INT
        SET @x=1
        WHILE (@x <= @Limite)
                BEGIN
                        IF (@x % 2)=0
                                PRINT @x
                        SET @x += 1
                END
```



Stored Procedure (Estrutura de Repetição WHILE...)

```
CREATE OR ALTER PROCEDURE SP_Contar
      @Limite INT, @Paridade CHAR(1)
AS
      IF (@Paridade != 'P' AND @Paridade != 'I')
          PRINT 'Parâmetro desconhecido!'
      ELSE
          BEGIN
               DECLARE @x INT
               SET @x=1
               WHILE (@x <= @Limite)
                     BEGIN
                          IF (@Paridade='P' AND (@x % 2)=0 ) OR
                             (@Paridade='I' AND (@x % 2)!=0)
                              PRINT @x
                          SET @x += 1
                     END;
          END;
```

```
EXECUTE SP_Contar 10, 'P';
```

Stored Procedures

(Exercícios)

Prof. Me. Fred Lucena

Stored Procedure (Exercício 01)

De posse de uma tabela chamada **TESTE** com um campo **'NOME' VARCHAR**(30), desenvolva um Stored Procedure que solicite uma quantidade qualquer e insira no arquivo teste, 'NOME 1', 'NOME 2',..., 'NOME N'.

Stored Procedure (Exercício 01)

```
CREATE OR ALTER PROCEDURE SP_Inserir
        @Limite INT
AS
        DECLARE @x INT = 1
        WHILE (@x <= @Limite)
                BEGIN
                        INSERT INTO teste
                        VALUES ('Nome ' + CAST(@x AS VARCHAR))
                        SET @x += 1
                END;
```

Stored Procedure (Exercício 01)

```
CREATE OR ALTER PROCEDURE SP_Inserir
        @Limite INT
AS
        DECLARE @x INT = 1
        WHILE (@x <= @Limite)
                BEGIN
                        INSERT INTO teste
                        VALUES (CONCAT('Nome ', @x))
                        SET @x += 1
                END;
```

Stored Procedure (Exercício 01)

```
CREATE OR ALTER PROCEDURE SP_Inserir
        @Limite INT
AS
        DECLARE @x INT = 1 + (SELECT COUNT(*) FROM teste)
        SET @Limite += @x - 1
        WHILE (@x <= @Limite)
                BEGIN
                        INSERT INTO teste
                        VALUES (CONCAT('Nome ', @x))
                        SET @x += 1
                END;
```

Stored Procedure (Exercício 01)

```
CREATE OR ALTER PROCEDURE SP_Inserir
      @Limite INT
AS
      DECLARE @x INT = (SELECT TOP(1)
                        CAST(SUBSTRING(nome,6,10) AS INT) AS ordem
                        FROM teste
                        ORDER BY ordem DESC)
      SET @x = IIF(@x IS NULL,1,@x+1)
      SET @Limite = @Limite + @x
      WHILE (@x < @Limite)
            BEGIN
                  INSERT INTO teste
                        VALUES (CONCAT('Nome ', @x))
                  SET @x = @x + 1
            END;
```

Stored Procedure (Exercício 02)

Desenvolva um Stored Procedure que receba um número e escreva quais, entre 2 e este número, são primos.

Stored Procedure (Exercício 02)

```
CREATE OR ALTER PROCEDURE SP_Primo
                @Limite INT
AS
                DECLARE @x INT, @y INT, @p BIT
                SET @x=2
                WHILE (@x <= @Limite)
                      BEGIN
                           SET @p = 1
                           SET @y = 2
                           WHILE (@y <= @x - 1)
                                 BEGIN
                                      IF (@x % @y) = 0
                                         BEGIN
                                              SET @p = 0
                                              BREAK
                                         END
                                      SET @y = @y + 1
                                 END
                           IF @p = 1 PRINT CAST(@X AS VARCHAR) + ' é primo.'
                           SET @x = @x + 1
                      END
```

Function

Prof. Me. Fred Lucena

O que é uma function?

É um procedimento composto de comandos T-SQL com o objetivo de executar determinada tarefa e retornando um resultado pretendido. Fica armazenado no Banco de Dados e permite executar várias tarefas previamente definidas.

Sintaxe de uma function

```
CREATE OR ALTER  FUNCTION <nome_da_função> ( [parâmetros] )
        RETURNS <tipo de dado>
AS
        BEGIN
                <instruções>
                RETURN (<resultado>)
        END;
```

Chamada a uma function

```
SELECT dbo.<nome_da_função>( [parâmetros] );
```

Function (Exemplo 01)

```
CREATE FUNCTION F_Somar (@Num1 INT, @Num2 INT)
       RETURNS INT
AS
       BEGIN
              DECLARE @Soma INT
              SET @Soma = @Num1 + @Num2
              RETURN(@Soma)
       END;
```

```
SELECT dbo.F_Somar(10,2);
```

Function

(Retornando Tabela)

Prof. Me. Fred Lucena

Function (Exemplo 01)

```
CREATE FUNCTION F_Tabela ()
       RETURNS TABLE
AS
       RETURN(SELECT * FROM cadfun);
```

```
SELECT * FROM F_Tabela();
```

Function (Exercício 01)

Desenvolva uma função que solicite três notas, calcule a media aritmética e, de acordo com a tabela abaixo, devolva a situação do aluno.

Média < 5 -> REPROVADO

Média >=5 e Média < 7 -> RECUPERAÇÃO

Media >= 7 -> APROVADO

```
SELECT dbo.F_Situacao(5,8,7);
```

Function (Exemplo 01)

```
CREATE FUNCTION F_Situacao(@Nota1 FLOAT, @Nota2 FLOAT, @Nota3 FLOAT)
       RETURNS VARCHAR(11)
AS
       BEGIN
            DECLARE @Media FLOAT, @Retorno VARCHAR(11)
            SET @Media = (@Nota1+@Nota2+@Nota3)/3
            IF @Media < 5
               SET @Retorno = 'Reprovado'
            ELSE
               IF @Media < 7
                  SET @Retorno = 'Recuperação'
               ELSE
                  SET @Retorno = 'Aprovado'
            RETURN (@Retorno)
       END;
```

```
SELECT dbo.F_Situacao(5,8,7);
```

Function (Exemplo 02)

```
ALTER FUNCTION F_Situacao(@Nota1 FLOAT,@Nota2 FLOAT,@Nota3 FLOAT)
      RETURNS VARCHAR(11)
AS
      BEGIN
            DECLARE @Media FLOAT, @Retorno VARCHAR(20)
            SET @Media = (@Nota1+@Nota2+@Nota3)/3
            SET @Retorno = CASE
                              WHEN @Media < 5 THEN 'Reprovado'
                              WHEN @Media < 7 THEN 'Recuperação'
                              ELSE 'Aprovado'
                           END
            RETURN (@Retorno)
      END;
```

```
SELECT dbo.F_Situacao(5,8,7);
```

Function (Exemplo 03)

```
ALTER FUNCTION F_Situacao(@Nota1 FLOAT,@Nota2 FLOAT,@Nota3 FLOAT)
      RETURNS VARCHAR(11)
AS
      BEGIN
            DECLARE @Media FLOAT = (@Nota1+@Nota2+@Nota3)/3
            RETURN(CASE
                        WHEN @Media < 5 THEN 'Reprovado'
                        WHEN @Media < 7 THEN 'Recuperação'
                        ELSE 'Aprovado'
                  END)
      END;
```

```
SELECT dbo.F_Situacao(5,8,7);
```

Function (Exemplo 04)

```
ALTER FUNCTION F_Situacao(@Nota1 FLOAT,@Nota2 FLOAT,@Nota3 FLOAT)
      RETURNS VARCHAR(11)
AS
      BEGIN
            DECLARE @Media FLOAT = (@Nota1+@Nota2+@Nota3)/3
            RETURN(IIF(@Media < 5,'Reprovado',IIF(@Media < 7,'Recuperação','Aprovado')))
      END;
```

```
SELECT dbo.F_Situacao(5,8,7);
```

Function (Exemplo 05)

```
ALTER FUNCTION F_Situacao(@Nota1 FLOAT,@Nota2 FLOAT,@Nota3 FLOAT)
      RETURNS VARCHAR(11)
AS
      BEGIN
           RETURN(IIF((@Nota1+@Nota2+@Nota3)/3 < 5,'Reprovado',IIF((@Nota1+@Nota2+@Nota3)/3 < 7,'Recuperação','Aprovado')))
      END;
```

```
SELECT dbo.F_Situacao(5,8,7);
```

Function (Exercício 02)

Crie e alimente a tabela ALUNOS conforme modelo abaixo.

ALUNOS			
NOME	**NOTA1**	**NOTA2**	**NOTA3**
Pedro	2,0	7,0	4,0
Carlos	4,0	9,0	6,0
Ana	6,0	1,0	8,0
Paulo	8,0	3,0	10,0
Maria	10,0	5,0	9,0
José	1,0	7,0	8,0
João	3,0	9,0	7,0
Cristina	5,0	2,0	6,0

Function (Exercício 02)

Faça a consulta sugerida abaixo.

```
SELECT nome  AS 'NOME',
       nota1 AS 'NOTA1',
       nota2 AS 'NOTA2',
       nota3 AS 'NOTA3',
       CAST((nota1+nota2+nota3)/3 AS DECIMAL(3,1)) AS 'MÉDIA',
       dbo.F_Situacao(nota1,nota2,nota3)  AS 'SITUAÇÃO'
FROM Alunos
```

View

(Criando Visões)

Prof. Me. Fred Lucena

Criando uma View

```
CREATE VIEW Vcadfun
AS
SELECT nome, funcao  FROM cadfun;
```

Tabelas Temporárias

(Criando Tabelas Temporárias)

Prof. Me. Fred Lucena

Criando uma Tabela Temporária

```
CREATE TABLE #Agenda ( Telefone CHAR(10),
                                    Nome     VARCHAR(50) );
INSERT INTO #Agenda
        VALUES ('1234-5678', 'Pedro'),
                        ('2345-6789', 'Paulo'),
                        ('3456-7890', 'Mário');

SELECT * FROM #Agenda;
```